

TVM: An Automated End-to-End Optimizing Compiler for Deep Learning

Tianqi Chen¹, Thierry Moreau¹, Ziheng Jiang^{1,2}, Lianmin Zheng³, Eddie Yan¹

Meghan Cowan¹, Haichen Shen¹, Leyuan Wang^{4,2}, Yuwei Hu⁵, Luis Ceze¹, Carlos Guestrin¹, Arvind Krishnamurthy¹
¹Paul G. Allen School of Computer Science & Engineering, University of Washington

² AWS, ³Shanghai Jiao Tong University, ⁴UC Davis, ⁵Cornell

Abstract

There is an increasing need to bring machine learning to a wide diversity of hardware devices. Current frameworks rely on vendor-specific operator libraries and optimize for a narrow range of server-class GPUs. Deploying workloads to new platforms – such as mobile phones, embedded devices, and accelerators (e.g., FPGAs, ASICs) – requires significant manual effort. We propose TVM, a compiler that exposes graph-level and operator-level optimizations to provide performance portability to deep learning workloads across diverse hardware back-ends. TVM solves optimization challenges specific to deep learning, such as high-level operator fusion, mapping to arbitrary hardware primitives, and memory latency hiding. It also automates optimization of low-level programs to hardware characteristics by employing a novel, learning-based cost modeling method for rapid exploration of code optimizations. Experimental results show that TVM delivers performance across hardware back-ends that are competitive with state-of-the-art, hand-tuned libraries for low-power CPU, mobile GPU, and server-class GPUs. We also demonstrate TVM’s ability to target new accelerator back-ends, such as the FPGA-based generic deep learning accelerator. The system is open sourced and in production use inside several major companies.

1 Introduction

Deep learning (DL) models can now recognize images, process natural language, and defeat humans in challenging strategy games. There is a growing demand to deploy smart applications to a wide spectrum of devices, ranging from cloud servers to self-driving cars and embedded devices. Mapping DL workloads to these devices is complicated by the diversity of hardware characteristics, including embedded CPUs, GPUs, FPGAs, and ASICs (e.g., the TPU [21]). These hardware targets diverge in

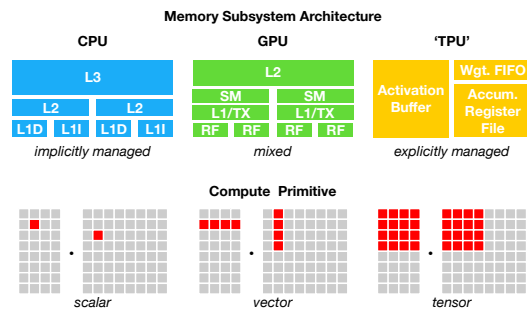


Figure 1: CPU, GPU and TPU-like accelerators require different on-chip memory architectures and compute primitives. This divergence must be addressed when generating optimized code.

terms of memory organization, compute functional units, etc., as shown in Figure 1.

Current DL frameworks, such as TensorFlow, MXNet, Caffe, and PyTorch, rely on a computational graph intermediate representation to implement optimizations, e.g., auto differentiation and dynamic memory management [3, 4, 9]. Graph-level optimizations, however, are often too high-level to handle hardware back-end-specific operator-level transformations. Most of these frameworks focus on a narrow class of server-class GPU devices and delegate target-specific optimizations to highly engineered and vendor-specific operator libraries. These operator-level libraries require significant manual tuning and hence are too specialized and opaque to be easily ported across hardware devices. Providing support in various DL frameworks for diverse hardware back-ends presently requires significant engineering effort. Even for supported back-ends, frameworks must make the difficult choice between: (1) avoiding graph optimizations that yield new operators not in the predefined operator library, and (2) using unoptimized implementations of these new operators.

To enable both graph- and operator-level optimiza-

TVM: 面向深度学习的自动化端到端优化编译器

陈天奇¹、蒂埃里·莫罗¹、蒋子恒^{1,2}、郑伶俐³、埃迪·严¹

梅根·考恩¹、沈海辰¹、王乐媛^{4,2}、胡宇伟⁵、路易斯·塞泽¹、卡洛斯·格斯特林¹、阿文德·克里希纳穆尔蒂¹ 华盛顿大学
保罗·G·艾伦计算机科学与工程学院

² AWS, ³上海交通大学, ⁴加州大学戴维斯分校, ⁵康奈尔大学

摘要

将机器学习应用于各种硬件设备的需求日益增长。当前框架依赖于特定供应商的算子库，并针对有限范围的服务器级GPU进行优化。将工作负载部署到新平台——如手机、嵌入式设备和加速器（例如FPGA、ASIC）——需要大量人工操作。我们提出了TVM，这是一种编译器，它通过图级和算子级优化，为不同硬件后端上的深度学习工作负载提供性能可移植性。TVM解决了深度学习特有的优化挑战，例如高级算子融合、映射到任意硬件原语以及内存延迟隐藏。它还通过采用一种新颖的、基于学习的成本建模方法，快速探索代码优化，从而自动化地将低级程序优化到硬件特性。实验结果表明，TVM在各种硬件后端上提供的性能，可与针对低功耗CPU、移动GPU和服务级GPU的最先进手工调优库相媲美。我们还展示了TVM面向新型加速器后端的能力，例如基于FPGA的通用深度学习加速器。该系统已开源，并在多家主要公司内部投入生产使用。

1 引言

深度学习（DL）模型现已能够识别图像、处理自然语言，并在具有挑战性的策略游戏中战胜人类。将智能应用部署到从云服务器到自动驾驶汽车及嵌入式设备的广泛设备需求日益增长。由于硬件特性的多样性，包括嵌入式CPU、GPU、FPGA和ASIC（例如TPU[21]），将DL工作负载映射到这些设备变得复杂。这些硬件目标在

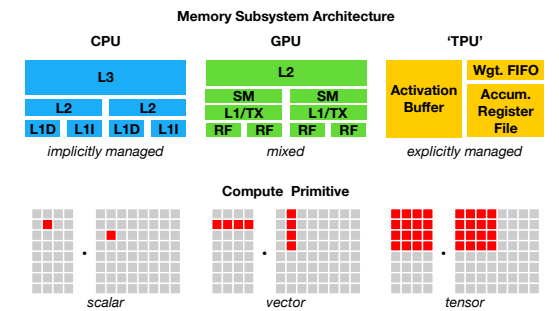


图1: CPU、GPU和类TPU加速器需要不同的片上存储器架构和计算原语。在生成优化代码时，必须解决这种差异。

在内存组织、计算功能单元等方面，如图1所示。

当前的深度学习框架，如TensorFlow、MXNet、Caffe和PyTorch，依赖于一种称为计算图的中间表示来实现优化，例如自动微分和动态内存管理[3, 4, 9]。然而，图级优化往往过于高层，难以处理硬件后端特定的算子级转换。这些框架大多专注于服务器级GPU设备的狭窄类别，并将特定目标的优化委托给高度工程化且供应商特定的算子库。这些算子级库需要大量手动调优，因此过于专门化和不透明，难以在不同硬件设备间轻松移植。目前，为各种硬件后端在深度学习框架中提供支持需要大量的工程努力。即使对于已支持的后端，框架也必须在以下两者之间做出艰难选择：(1) 避免产生预定义算子库中不存在的新算子的图优化，以及(2) 使用这些新算子的未优化实现。

为了实现图级和算子级的双重优化

tions for diverse hardware back-ends, we take a fundamentally different, end-to-end approach. *We built TVM, a compiler that takes a high-level specification of a deep learning program from existing frameworks and generates low-level optimized code for a diverse set of hardware back-ends.* To be attractive to users, TVM needs to offer performance competitive with the multitude of manually optimized operator libraries across diverse hardware back-ends. This goal requires addressing the key challenges described below.

Leveraging Specific Hardware Features and Abstractions. DL accelerators introduce optimized tensor compute primitives [1, 12, 21], while GPUs and CPUs continuously improve their processing elements. This poses a significant challenge in generating optimized code for a given operator description. The inputs to hardware instructions are multi-dimensional, with fixed or variable lengths; they dictate different data layouts; and they have special requirements for memory hierarchy. The system must effectively exploit these complex primitives to benefit from acceleration. Further, accelerator designs also commonly favor leaner control [21] and offload most scheduling complexity to the compiler stack. For specialized accelerators, the system now needs to generate code that explicitly controls pipeline dependencies to hide memory access latency – a job that hardware performs for CPUs and GPUs.

Large Search Space for Optimization Another challenge is producing efficient code without manually tuning operators. The combinatorial choices of memory access, threading pattern, and novel hardware primitives creates a huge configuration space for generated code (e.g., loop tiles and ordering, caching, unrolling) that would incur a large search cost if we implement black box auto-tuning. One could adopt a predefined cost model to guide the search, but building an accurate cost model is difficult due to the increasing complexity of modern hardware. Furthermore, such an approach would require us to build separate cost models for each hardware type.

TVM addresses these challenges with three key modules. (1) We introduce a *tensor expression language* to build operators and provide program transformation primitives that generate different versions of the program with various optimizations. This layer extends Halide [32]’s compute/schedule separation concept by also separating target hardware intrinsics from transformation primitives, which enables support for novel accelerators and their corresponding new intrinsics. Moreover, we introduce new transformation primitives to address GPU-related challenges and enable deployment to specialized accelerators. We can then apply different sequences of program transformations to form a rich space

of valid programs for a given operator declaration. (2) We introduce an *automated program optimization framework* to find optimized tensor operators. The optimizer is guided by an ML-based cost model that adapts and improves as we collect more data from a hardware back-end. (3) On top of the automatic code generator, we introduce a *graph rewriter* that takes full advantage of high- and operator-level optimizations.

By combining these three modules, TVM can take model descriptions from existing deep learning frameworks, perform joint high- and low-level optimizations, and generate hardware-specific optimized code for back-ends, e.g., CPUs, GPUs, and FPGA-based specialized accelerators.

This paper makes the following contributions:

- We identify the major optimization challenges in providing performance portability to deep learning workloads across diverse hardware back-ends.
- We introduce novel schedule primitives that take advantage of cross-thread memory reuse, novel hardware intrinsics, and latency hiding.
- We propose and implement a machine learning based optimization system to automatically explore and search for optimized tensor operators.
- We build an end-to-end compilation and optimization stack that allows the deployment of deep learning workloads specified in high-level frameworks (including TensorFlow, MXNet, PyTorch, Keras, CNTK) to diverse hardware back-ends (including CPUs, server GPUs, mobile GPUs, and FPGA-based accelerators). The open-sourced TVM is in production use inside several major companies.

We evaluated TVM using real world workloads on a server-class GPU, an embedded GPU, an embedded CPU, and a custom generic FPGA-based accelerator. Experimental results show that TVM offers portable performance across back-ends and achieves speedups ranging from $1.2\times$ to $3.8\times$ over existing frameworks backed by hand-optimized libraries.

2 Overview

This section describes TVM by using an example to walk through its components. Figure 2 summarizes execution steps in TVM and their corresponding sections in the paper. The system first takes as input a model from an existing framework and transforms it into a computational graph representation. It then performs high-level dataflow rewriting to generate an optimized graph. The operator-level optimization module must generate efficient code for each fused operator in this graph. Operators are specified in a declarative tensor expression lan-

为了适应多样化的硬件后端，我们采取了一种根本不同的端到端方法。 *We built*

TVM, a compiler that takes a high-level specification of a deep learning program from existing frameworks and generates low-level optimized code for a diverse set of hardware back-ends. 为了吸引用户，TVM需要与各种硬件后端上众多手动优化的算子库相竞争的性能。这一目标需要解决以下关键挑战。

利用特定硬件特性和抽象。深度学习加速器引入了优化的张量计算原语[1, 12, 21]，而GPU和CPU则持续改进其处理单元。这为给定算子描述生成优化代码带来了重大挑战。硬件指令的输入是多维的，具有固定或可变长度；它们规定了不同的数据布局；并对内存层次结构有特殊要求。系统必须有效利用这些复杂的原语才能从加速中获益。此外，加速器设计通常倾向于更精简的控制[21]，并将大部分调度复杂性卸载到编译器栈。对于专用加速器，系统现在需要生成能显式控制流水线依赖关系以隐藏内存访问延迟的代码——这项工作CPU和GPU上是由硬件完成的。

优化的大搜索空间另一个挑战是在无需手动调整运算符的情况下生成高效代码。内存访问、线程模式以及新型硬件原语的组合选择为生成的代码（例如，循环分块与排序、缓存、展开）创造了巨大的配置空间，如果我们实施黑盒自动调优，将产生巨大的搜索成本。可以采用预定义的成本模型来指导搜索，但由于现代硬件的复杂性日益增加，构建精确的成本模型十分困难。此外，这种方法需要我们为每种硬件类型构建独立的成本模型。

TVM通过三个关键模块应对这些挑战。(1)我们引入*tensor expression language*来构建算子，并提供程序转换原语，以生成具有不同优化版本的程序。该层扩展了Halide[32]的计算/调度分离概念，同时将目标硬件内联函数与转换原语分离，从而支持新型加速器及其对应的新内联函数。此外，我们引入新的转换原语以解决GPU相关挑战，并实现向专用加速器的部署。随后，我们可以应用不同的程序转换序列来构建丰富的空间

对于给定运算符声明的有效程序。(2)我们引入一个*automated program optimization framework*来寻找优化的张量运算符。该优化器由基于机器学习的成本模型指导，该模型随着我们从硬件后端收集更多数据而不断适应和改进。(3)在自动代码生成器之上，我们引入了一个*graph rewriter*，它充分利用了高级和运算符级别的优化。

通过整合这三个模块，TVM能够接收来自现有深度学习框架的模型描述，执行联合的高层与底层优化，并为后端（如CPU、GPU及基于FPGA的专用加速器）生成针对特定硬件优化的代码。

本文作出以下贡献：

- 我们确定了在跨不同硬件后端为深度学习工作负载提供性能可移植性方面的主要优化挑战。
- 我们引入了新的调度原语，这些原语能够利用跨线程内存重用、新型硬件内置功能以及延迟隐藏技术。
- 我们提出并实现了一个基于机器学习的优化系统，用于自动探索和搜索优化的张量算子。
- 我们构建了一个端到端的编译和优化栈，使得能够将高级框架（包括TensorFlow、MXNet、PyTorch、Keras、CNTK）中指定的深度学习工作负载部署到多样化的硬件后端（包括CPU、服务器GPU、移动GPU以及基于FPGA的加速器）。开源的TVM已在多家主要公司内部投入生产使用。

我们在服务器级GPU、嵌入式GPU、嵌入式CPU以及基于FPGA的定制通用加速器上，使用真实世界的工作负载对TVM进行了评估。实验结果表明，TVM在不同后端之间具备可移植的性能表现，相较于依赖手动优化库的现有框架，其加速比达到 $1.2\times$ 至 $3.8\times$ 。

2 概述

本节通过一个示例来介绍TVM的各个组件。图2总结了TVM的执行步骤及其在论文中对应的章节。系统首先接收来自现有框架的模型作为输入，并将其转换为计算图表示。接着执行高级数据流重写以生成优化后的计算图。算子级优化模块必须为该图中每个融合算子生成高效代码。算子通过声明式张量表达式语言进行描述——

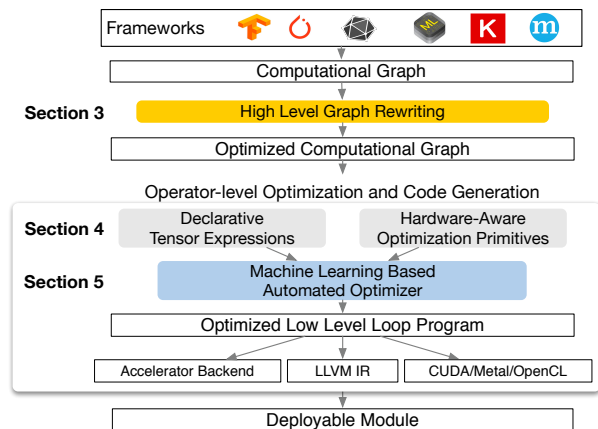


Figure 2: System overview of TVM. The current stack supports descriptions from many deep learning frameworks and exchange formats, such as CoreML and ONNX, to target major CPU, GPU and specialized accelerators.

guage; execution details are unspecified. TVM identifies a collection of possible code optimizations for a given hardware target's operators. Possible optimizations form a large space, so we use an ML-based cost model to find optimized operators. Finally, the system packs the generated code into a deployable module.

End-User Example. In a few lines of code, a user can take a model from existing deep learning frameworks and call the TVM API to get a deployable module:

```
import tvml as t
# Use keras framework as example, import model
graph, params = t.frontend.from_keras(keras_model)
target = t.target.cuda()
graph, lib, params = t.compiler.build(graph, target, params)
```

This compiled runtime module contains three components: the final optimized computational graph (graph), generated operators (lib), and module parameters (params). These components can then be used to deploy the model to the target back-end:

```
import tvml.runtime as t
module = runtime.create(graph, lib, t.cuda(0))
module.set_input(**params)
module.run(data=data_array)
output = tvml.nd.empty(out_shape, ctx=t.cuda(0))
module.get_output(0, output)
```

TVM supports multiple deployment back-ends in languages such as C++, Java and Python. The rest of this paper describes TVM's architecture and how a system programmer can extend it to support new back-ends.

3 Optimizing Computational Graphs

Computational graphs are a common way to represent programs in DL frameworks [3, 4, 7, 9]. Figure 3 shows

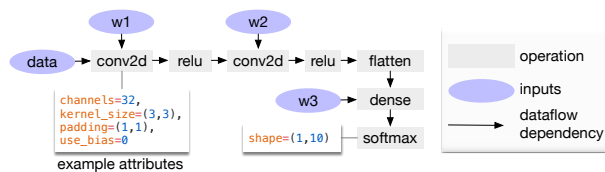


Figure 3: Example computational graph of a two-layer convolutional neural network. Each node in the graph represents an operation that consumes one or more tensors and produces one or more tensors. Tensor operations can be parameterized by attributes to configure their behavior (e.g., padding or strides).

an example computational graph representation of a two-layer convolutional neural network. The main difference between this high-level representation and a low-level compiler intermediate representation (IR), such as LLVM, is that the intermediate data items are large, multi-dimensional tensors. Computational graphs provide a global view of operators, but they avoid specifying how each operator must be implemented. Like LLVM IRs, a computational graph can be transformed into functionally equivalent graphs to apply optimizations. We also take advantage of shape specificity in common DL workloads to optimize for a fixed set of input shapes.

TVM exploits a computational graph representation to apply high-level optimizations: a node represents an operation on tensors or program inputs, and edges represent data dependencies between operations. It implements many graph-level optimizations, including: *operator fusion*, which fuses multiple small operations together; *constant-folding*, which pre-computes graph parts that can be determined statically, saving execution costs; a *static memory planning pass*, which pre-allocates memory to hold each intermediate tensor; and *data layout transformations*, which transform internal data layouts into back-end-friendly forms. We now discuss operator fusion and the data layout transformation.

Operator Fusion. Operator fusion combines multiple operators into a single kernel without saving the intermediate results in memory. This optimization can greatly reduce execution time, particularly in GPUs and specialized accelerators. Specifically, we recognize four categories of graph operators: (1) injective (one-to-one map, e.g., add), (2) reduction (e.g., sum), (3) complex-out-fusable (can fuse element-wise map to output, e.g., conv2d), and (4) opaque (cannot be fused, e.g., sort). We provide generic rules to fuse these operators, as follows. Multiple injective operators can be fused into another injective operator. A reduction operator can be fused with input injective operators (e.g., fuse scale and sum). Operators such as conv2d are complex-out-fusable, and we

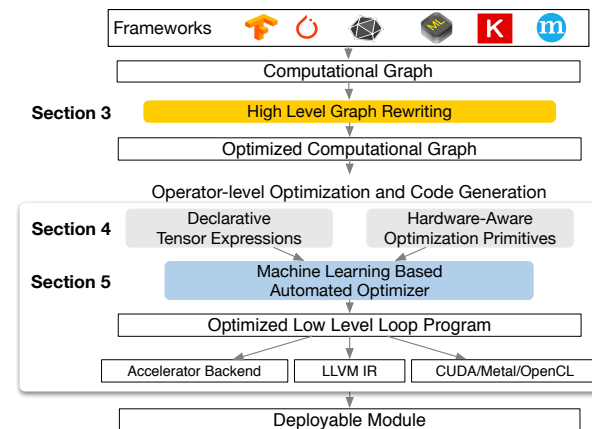


图2: TVM系统概览。当前堆栈支持从多种深度学习框架和交换格式（如CoreML和ONNX）进行描述，以面向主流CPU、GPU及专用加速器。

语言；执行细节未指定。TVM为给定硬件目标的算子识别出一系列可能的代码优化。可能的优化构成了一个巨大的空间，因此我们使用基于机器学习的成本模型来寻找优化后的算子。最后，系统将生成的代码打包成可部署的模块。

终端用户示例。只需几行代码，用户就能从现有深度学习框架中获取模型，并调用TVM API获得可部署模块：

```
import tvml as t # 以keras框架为例，导入模型图、参数 = t.frontend.from_keras(keras_model)
目标 = t.target.cuda() 图、库、参数 = t.compiler.build(graph, target, params)
```

该编译后的运行时模块包含三个组件：最终优化的计算图（graph）、生成的算子（lib）以及模块参数（params）。这些组件随后可用于将模型部署到目标后端：

```
导入 tvml.runtime 作为 t 模块 runtime.create(图, 库, t.cuda(0)) 模块
set_input(**参数) 模块on(数据=数据数组) 输出 tvml.nd.empty(输出形状, 上下文=t.cuda(0)) 模块t_output(0, 输出)
```

TVM支持多种部署后端，如C++、Java和Python语言环境。本文其余部分将阐述TVM的架构，以及系统程序员如何扩展它以支持新的后端。

3 优化计算图

计算图是深度学习框架中表示程序的常用方法[3, 4, 7, 9]。图3展示了

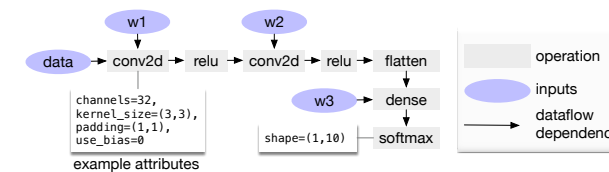


图3: 一个两层卷积神经网络的计算图示例。图中的每个节点代表一个操作，该操作消耗一个或多个张量并产生一个或多个张量。张量操作可以通过属性进行参数化，以配置其行为（例如，填充或步幅）。

一个双层卷积神经网络的计算图表示示例。这种高层表示与低层编译器中间表示（IR）（如LLVM）的主要区别在于，中间数据项是大型的多维张量。计算图提供了算子的全局视图，但避免了指定每个算子必须如何实现。与LLVM IR类似，计算图可以转换为功能等效的图以应用优化。我们还利用常见深度学习工作负载中的形状特异性，针对一组固定的输入形状进行优化。

TVM利用计算图表示来应用高级优化：节点代表张量或程序输入上的操作，边代表操作间的数据依赖关系。它实现了多种图级优化，包括：*operator fusion*，将多个小型操作融合在一起；*constant-folding*，预计算可静态确定的图部分以节省执行成本；*static memory planning pass*，预分配内存以存储每个中间张量；以及*data layout transformations*，将内部数据布局转换为后端友好的形式。接下来我们将讨论算子融合与数据布局转换。

算子融合。算子融合将多个算子合并为单个内核，无需将中间结果保存到内存中。这种优化可以大大减少执行时间，尤其是在GPU和专用加速器中。具体来说，我们将图算子分为四类：(1) 单射（一对一映射，例如加法），(2) 归约（例如求和），(3) 输出可融合复杂算子（可将逐元素映射融合到输出，例如conv2d），以及(4) 不透明算子（无法融合，例如排序）。我们提供以下通用规则来融合这些算子。多个单射算子可以融合为另一个单射算子。归约算子可以与输入单射算子融合（例如融合缩放和求和）。像conv2d这样的算子是输出可融合复杂算子，我们

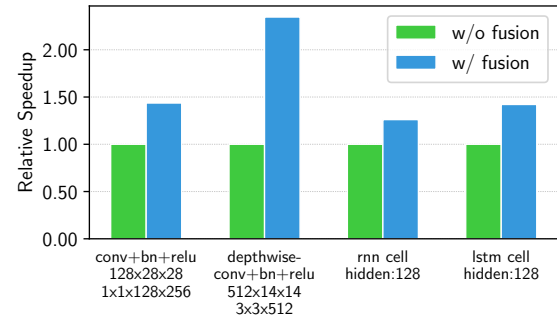


Figure 4: Performance comparison between fused and non-fused operations. TVM generates both operations. Tested on NVIDIA Titan X.

can fuse element-wise operators to its output. We can apply these rules to transform the computational graph into a fused version. Figure 4 demonstrates the impact of this optimization on different workloads. We find that fused operators generate up to a $1.2\times$ to $2\times$ speedup by reducing memory accesses.

Data Layout Transformation. There are multiple ways to store a given tensor in the computational graph. The most common data layout choices are column major and row major. In practice, we may prefer to use even more complicated data layouts. For instance, a DL accelerator might exploit 4×4 matrix operations, requiring data to be tiled into 4×4 chunks to optimize for access locality.

Data layout optimization converts a computational graph into one that can use better internal data layouts for execution on the target hardware. It starts by specifying the preferred data layout for each operator given the constraints dictated by memory hierarchies. We then perform the proper layout transformation between a producer and a consumer if their preferred data layouts do not match.

While high-level graph optimizations can greatly improve the efficiency of DL workloads, they are only as effective as what the operator library provides. Currently, the few DL frameworks that support operator fusion require the operator library to provide an implementation of the fused patterns. With more network operators introduced on a regular basis, the number of possible fused kernels can grow dramatically. This approach is no longer sustainable when targeting an increasing number of hardware back-ends since the required number of fused pattern implementations grows combinatorially with the number of data layouts, data types, and accelerator intrinsics that must be supported. It is not feasible to handcraft operator kernels for the various operations desired by a program and for each back-end. To

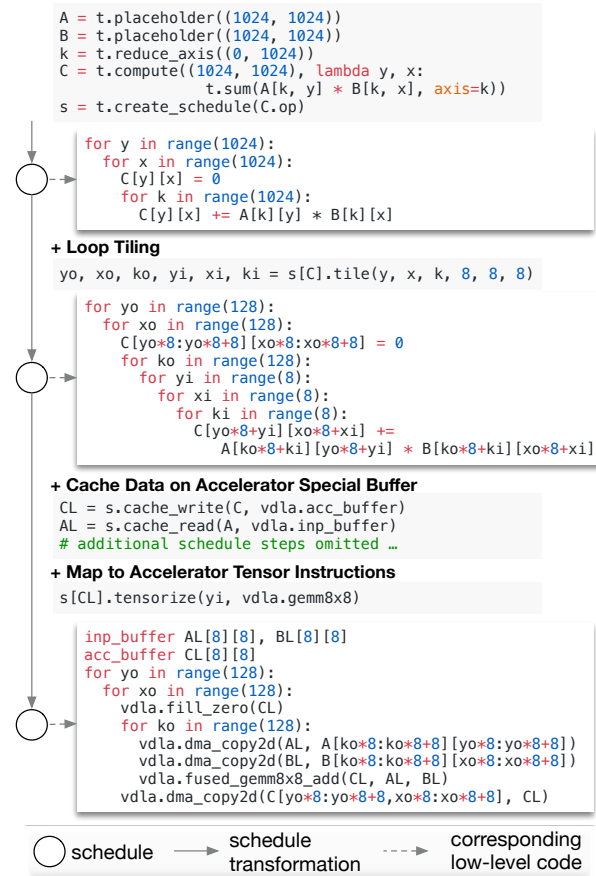


Figure 5: Example schedule transformations that optimize a matrix multiplication on a specialized accelerator.

this end, we next propose a code generation approach that can generate various possible implementations for a given model’s operators.

4 Generating Tensor Operations

TVM produces efficient code for each operator by generating many valid implementations on each hardware back-end and choosing an optimized implementation. This process builds on Halide’s idea of decoupling descriptions from computation rules (or *schedule optimizations*) [32] and extends it to support new optimizations (nested parallelism, tensorization, and latency hiding) and a wide array of hardware back-ends. We now highlight TVM-specific features.

4.1 Tensor Expression and Schedule Space

We introduce a tensor expression language to support automatic code generation. Unlike high-level computation graph representations, where the implementation of tensor operations is opaque, each operation is described in

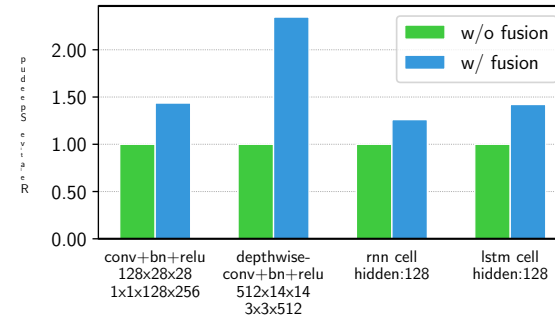


图4：融合操作与非融合操作的性能对比。TVM生成了两种操作。测试平台为NVIDIA Titan X。

可以将逐元素操作符融合到其输出中。我们可以应用这些规则将计算图转换为融合版本。图4展示了这种优化对不同工作负载的影响。我们发现，通过减少内存访问，融合操作符能带来高达 $1.2\times$ 到 $2\times$ 的加速效果。

数据布局转换。在计算图中存储给定张量有多种方式。最常见的数据布局选择是列优先和行优先。在实践中，我们可能更倾向于使用更复杂的数据布局。例如，深度学习加速器可能利用 4×4 矩阵运算，这需要将数据分块成 4×4 的块以优化访问局部性。

数据布局优化将计算图转换为能够使用更优内部数据布局以在目标硬件上执行的图。它首先根据内存层次结构所规定的约束，为每个算子指定首选数据布局。如果生产者和消费者的首选数据布局不匹配，我们会在它们之间执行适当的布局转换。

虽然高级图优化能极大提升深度学习工作负载的效率，但其效果完全取决于算子库的支撑能力。当前少数支持算子融合的深度学习框架，都要求算子库为每种融合模式提供具体实现。随着网络算子持续定期增加，可能的融合内核数量将呈爆炸式增长。当需要适配日益增多的硬件后端时，这种模式已难以为继——因为所需融合模式实现数量会随着必须支持的数据布局、数据类型及加速器指令集的组合呈指数级增长。为不同程序需求及各类后端手工定制算子内核显然不可行。

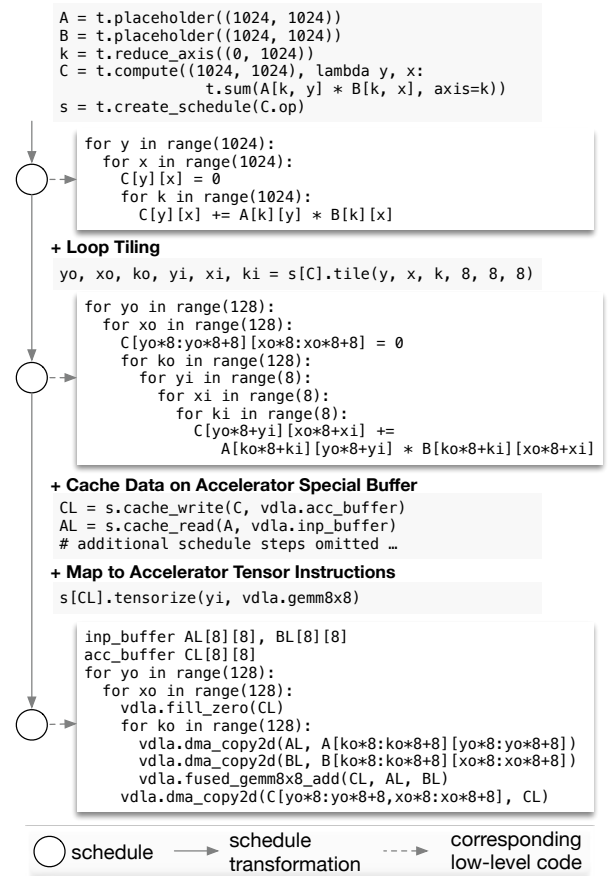


图5：在专用加速器上优化矩阵乘法的调度转换示例。

为此，我们接下来提出一种代码生成方法，能够为给定模型的算子生成多种可能的实现。

4 生成张量运算

TVM通过在每个硬件后端生成多种有效实现并选择优化实现，为每个算子生成高效代码。这一过程基于Halide将计算描述与计算规则（或 *schedule optimizations*）解耦的思想[32]，并将其扩展以支持新优化（嵌套并行、张量化及延迟隐藏）及广泛的硬件后端。以下重点介绍TVM的特有功能。

4.1 张量表达式与调度空间

我们引入了一种张量表达式语言来支持自动代码生成。与高层次计算图表示不同，在那种表示中张量运算的实现是不透明的，而我们的语言中每个运算都以

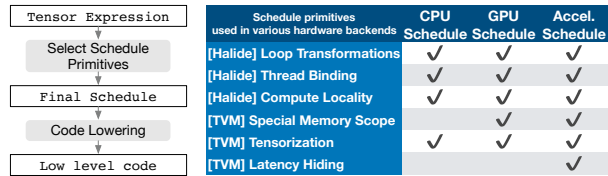


Figure 6: TVM schedule lowering and code generation process. The table lists existing Halide and novel TVM scheduling primitives being used to optimize schedules for CPUs, GPUs and accelerator back-ends. Tensorization is essential for accelerators, but it can also be used for CPUs and GPUs. Special memory-scope enables memory reuse in GPUs and explicit management of on-chip memory in accelerators. Latency hiding is specific to TPU-like accelerators.

an index formula expression language. The following code shows an example tensor expression to compute transposed matrix multiplication:

```
m, n, h = t.var('m'), t.var('n'), t.var('h')
A = t.placeholder((m, h), name='A')
B = t.placeholder((n, h), name='B')
k = t.reduce_axis((0, h), name='k')
C = t.compute((m, n), lambda y, x:
    t.sum(A[k, y] * B[k, x], axis=k))
```

computing rule

result shape

Each compute operation specifies both the shape of the output tensor and an expression describing how to compute each element of it. Our tensor expression language supports common arithmetic and math operations and covers common DL operator patterns. The language does not specify the loop structure and many other execution details, and it provides flexibility for adding hardware-aware optimizations for various back-ends. Adopting the decoupled compute/schedule principle from Halide [32], we use a schedule to denote a specific mapping from a tensor expression to low-level code. Many possible schedules can perform this function.

We build a schedule by incrementally applying basic transformations (schedule primitives) that preserve the program’s logical equivalence. Figure 5 shows an example of scheduling matrix multiplication on a specialized accelerator. Internally, TVM uses a data structure to keep track of the loop structure and other information as we apply schedule transformations. This information can then help generate low-level code for a given final schedule.

Our tensor expression takes cues from Halide [32], Darkroom [17], and TACO [23]. Its primary enhancements include support for the new schedule optimizations discussed below. To achieve high performance on many back-ends, we must support enough schedule primitives to cover a diverse set of optimizations on different hardware back-ends. Figure 6 summarizes the operation code generation process and schedule primi-

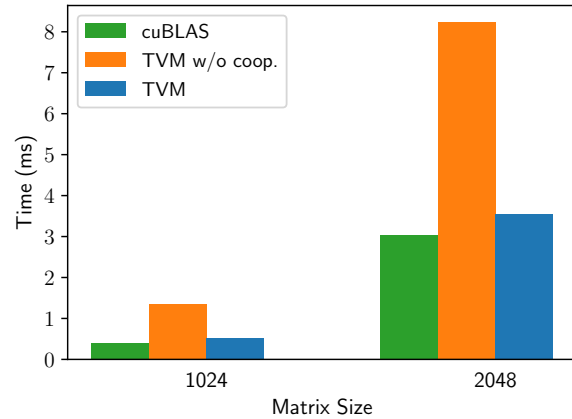


Figure 7: Performance comparison between TVM with and without cooperative shared memory fetching on matrix multiplication workloads. Tested on an NVIDIA Titan X.

tives that TVM supports. We reuse helpful primitives and the low-level loop program AST from Halide, and we introduce new primitives to optimize GPU and accelerator performance. The new primitives are necessary to achieve optimal GPU performance and essential for accelerators. CPU, GPU, TPU-like accelerators are three important types of hardware for deep learning. This section describes new optimization primitives for CPUs, GPUs and TPU-like accelerators, while section 5 explains how to automatically derive efficient schedules.

4.2 Nested Parallelism with Cooperation

Parallelism is key to improving the efficiency of compute-intensive kernels in DL workloads. Modern GPUs offer massive parallelism, requiring us to bake parallel patterns into schedule transformations. Most existing solutions adopt a model called *nested parallelism*, a form of fork-join. This model requires a parallel schedule primitive to parallelize a data parallel task; each task can be further recursively subdivided into subtasks to exploit the target architecture’s multi-level thread hierarchy (e.g., thread groups in GPU). We call this model *shared-nothing nested parallelism* because one working thread cannot look at the data of its sibling within the same parallel computation stage.

An alternative to the shared-nothing approach is to fetch data cooperatively. Specifically, groups of threads can cooperatively fetch the data they all need and place it into a shared memory space.¹ This optimization can take advantage of the GPU memory hierarchy and en-

¹ Halide recently added shared memory support but without general memory scope for accelerators.

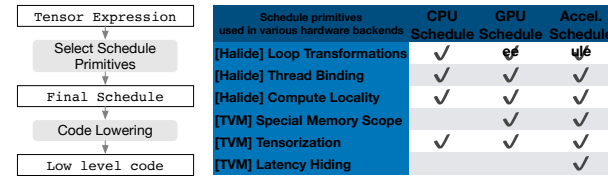


图6: TVM调度降低与代码生成流程。该表格列出了用于优化CPU、GPU及加速器后端调度的现有Halide及新型TVM调度原语。张量化对加速器至关重要，但也可用于CPU和GPU。专用内存作用域支持GPU中的内存重用及加速器中的片上内存显式管理。延迟隐藏特性专为类TPU加速器设计。

索引公式表达式语言。以下代码展示了一个计算转置矩阵乘法的张量表达式示例：

```
m, n, h = t.var('m'), t.var('n'), t.var('h')
A = t.placeholder((m, h), name='A')
B = t.placeholder((n, h), name='B')
k = t.reduce_axis((0, h), name='k')
C = t.compute((m, n), lambda y, x:
    t.sum(A[k, y] * B[k, x], axis=k))
```

computing rule

result shape

每个计算操作既指定输出张量的形状，也指定描述如何计算其每个元素的表达式。我们的张量表达式语言支持常见的算术和数学运算，并涵盖常见的深度学习算子模式。该语言不指定循环结构和许多其他执行细节，为各种后端添加硬件感知优化提供了灵活性。借鉴Halide[32]的解耦计算/调度原则，我们使用调度来表示从张量表达式到底层代码的具体映射。许多可能的调度都可以执行此功能。

我们通过逐步应用保持程序逻辑等价的基本变换（调度原语）来构建调度。图5展示了一个在专用加速器上调度矩阵乘法的示例。TVM内部使用一种数据结构来跟踪循环结构和其他信息，同时我们应用调度变换。这些信息随后可以帮助为给定的最终调度生成低级代码。

我们的张量表达式借鉴了Halide [32]、Darkroom [17]和TACO [23]的设计思路。其主要改进包括支持下文讨论的新型调度优化技术。为了在多种后端实现高性能，我们必须提供足够的调度原语，以覆盖不同硬件后端上的多样化优化策略。图6总结了操作代码生成过程及调度原语——

¹ Halide recently added shared memory support but without general memory scope for accelerators.

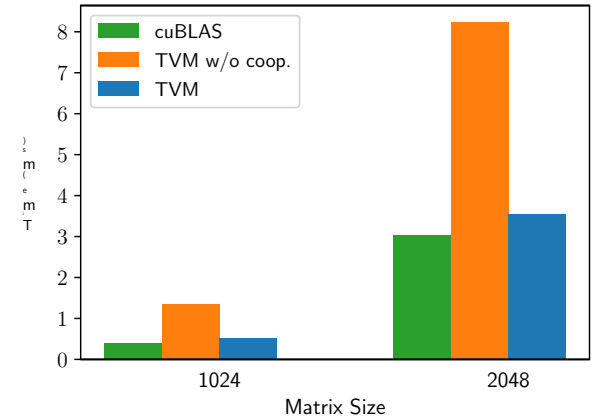


图7: TVM在使用与不使用协作共享内存获取机制下，在矩阵乘法工作负载上的性能对比。测试平台为NVIDIA Titan X。

TVM支持的原语。我们复用了Halide中的有用原语和底层循环程序AST，并引入了新的原语以优化GPU和加速器性能。这些新原语对于实现最佳GPU性能是必要的，对加速器也至关重要。CPU、GPU、类TPU加速器是深度学习的三种重要硬件类型。本节描述了针对CPU、GPU和类TPU加速器的新优化原语，而第5节将阐述如何自动推导高效调度方案。

4.2 协作式嵌套并行

并行性是提升深度学习工作负载中计算密集型内核效率的关键。现代GPU提供大规模并行能力，这要求我们将并行模式融入调度变换中。现有方案大多采用一种名为*nested parallelism*的模型，即一种分叉-合并形式。该模型需要一个并行调度原语来实现数据并行任务的并行化；每个任务可进一步递归细分为子任务，以充分利用目标架构的多级线程层次结构（例如GPU中的线程组）。我们将此模型称为*shared-nothing nested parallelism*，因为在同一并行计算阶段，工作线程无法访问其同级线程的数据。

共享内存方法的一种替代方案是协作式获取数据。具体来说，线程组可以协作获取它们共同需要的数据，并将其放入共享内存空间。¹这种优化能够利用GPU内存层次结构并提

¹ Halide recently added shared memory support but without general memory scope for accelerators.

able data reuse across threads through shared memory regions. TVM supports this well-known GPU optimization using a schedule primitive to achieve optimal performance. The following GPU code example optimizes matrix multiplication.

```
for thread_group (by, bx) in cross(64, 64):
    for thread_item (ty, tx) in cross(2, 2):
        local CL[8][8] = 0
        shared AS[2][8], BS[2][8]
        for k in range(1024):
            for i in range(4):
                AS[ty][i*4+tx] = A[k][by*64+ty*8+i*4+tx]
            for each i in 0..4:
                BS[ty][i*4+tx] = B[k][bx*64+ty*8+i*4+tx]
            memory_barrier_among_threads()
            for yi in range(8):
                for xi in range(8):
                    CL[yi][xi] += AS[yi] * BS[xi]
            for yi in range(8):
                for xi in range(8):
                    C[yo*8+yi][xo*8+xi] = CL[yi][xi]
```

All threads cooperatively load AS and BS in different parallel patterns

Barrier inserted automatically by compiler

Figure 7 demonstrates the impact of this optimization. We introduce the concept of *memory scopes* to the schedule space so that a compute stage (AS and BS in the code) can be marked as shared. Without explicit memory scopes, automatic scope inference will mark compute stages as thread-local. The shared task must compute the dependencies of all working threads in the group. Additionally, memory synchronization barriers must be properly inserted to guarantee that shared loaded data is visible to consumers. Finally, in addition to being useful to GPUs, memory scopes let us tag special memory buffers and create special lowering rules when targeting specialized DL accelerators.

4.3 Tensorization

DL workloads have high arithmetic intensity, which can typically be decomposed into tensor operators like matrix-matrix multiplication or 1D convolution. These natural decompositions have led to the recent trend of adding tensor compute primitives [1, 12, 21]. These new primitives create both opportunities and challenges for schedule-based compilation; while using them can improve performance, the compilation framework must seamlessly integrate them. We dub this *tensorization*: it is analogous to vectorization for SIMD architectures but has significant differences. Instruction inputs are multi-dimensional, with fixed or variable lengths, and each has different data layouts. More importantly, we cannot support a fixed set of primitives since new accelerators are emerging with their own variations of tensor instructions. We therefore need an *extensible* solution.

We make tensorization extensible by separating the target hardware intrinsic from the schedule with a mechanism for tensor-intrinsic declaration. We use the same tensor expression language to declare both the behavior of each new hardware intrinsic and the lowering rule associated with it. The following code shows how to declare an 8×8 tensor hardware intrinsic.

```
w, x = t.placeholder((8, 8)), t.placeholder((8, 8))
k = t.reduce_axis((0, 8))
y = t.compute((8, 8), lambda i, j:
    t.sum(w[i, k] * x[j, k], axis=k))

def gemm_intrin_lower(inputs, outputs):
    ww_ptr = inputs[0].access_ptr("r")
    xx_ptr = inputs[1].access_ptr("r")
    zz_ptr = outputs[0].access_ptr("w")
    compute = t.hardware_intrin("gemm8x8", ww_ptr, xx_ptr, zz_ptr)
    reset = t.hardware_intrin("fill_zero", zz_ptr)
    update = t.hardware_intrin("fuse_gemm8x8_add", ww_ptr, xx_ptr, zz_ptr)
    return compute, reset, update

gemm8x8 = t.decl_tensor_intrin(y.op, gemm_intrin_lower)
```

declare behavior

lowering rule to generate hardware intrinsics to carry out the computation

Additionally, we introduce a *tensorize* schedule primitive to replace a unit of computation with the corresponding intrinsics. The compiler matches the computation pattern with a hardware declaration and lowers it to the corresponding hardware intrinsic.

Tensorization decouples the schedule from specific hardware primitives, making it easy to extend TVM to support new hardware architectures. The generated code of tensorized schedules aligns with practices in high-performance computing: break complex operations into a sequence of micro-kernel calls. We can also use the *tensorize* primitive to take advantage of handcrafted micro-kernels, which can be beneficial in some platforms. For example, we implement ultra low precision operators for mobile CPUs that operate on data types that are one- or two-bits wide by leveraging a bit-serial matrix vector multiplication micro-kernel. This micro-kernel accumulates results into progressively larger data types to minimize the memory footprint. Presenting the micro-kernel as a tensor intrinsic to TVM yields up to a $1.5\times$ speedup over the non-tensorized version.

4.4 Explicit Memory Latency Hiding

Latency hiding refers to the process of overlapping memory operations with computation to maximize utilization of memory and compute resources. It requires different strategies depending on the target hardware back-end. On CPUs, memory latency hiding is achieved implicitly with simultaneous multithreading [14] or hardware prefetching [10, 20]. GPUs rely on rapid context switching of many warps of threads [44]. In contrast, specialized DL accelerators such as the TPU [21] usually favor leaner control with a *decoupled access-execute* (DAE) architecture [35] and offload the problem of fine-grained synchronization to software.

Figure 9 shows a DAE hardware pipeline that reduces runtime latency. Compared to a monolithic hardware design, the pipeline can hide most memory access overheads and almost fully utilize compute resources. To achieve higher utilization, the instruction stream must be augmented with fine-grained synchronization operations. Without them, dependencies cannot be enforced, leading to erroneous execution. Consequently, DAE hardware pipelines require fine-grained dependence enqueueing/dequeueing operations between the pipeline stages to guar-

通过共享内存区域实现跨线程的数据重用。TVM利用调度原语支持这一著名的GPU优化技术，以达到最佳性能。以下GPU代码示例优化了矩阵乘法。

```
for thread_group (by, bx) in cross(64, 64):
    for thread_item (ty, tx) in cross(2, 2):
        local CL[8][8] = 0
        shared AS[2][8], BS[2][8]
        for k in range(1024):
            for i in range(4):
                AS[ty][i*4+tx] = A[k][by*64+ty*8+i*4+tx]
            for each i in 0..4:
                BS[ty][i*4+tx] = B[k][bx*64+ty*8+i*4+tx]
            memory_barrier_among_threads()
            for yi in range(8):
                for xi in range(8):
                    CL[yi][xi] += AS[yi] * BS[xi]
            for yi in range(8):
                for xi in range(8):
                    C[yo*8+yi][xo*8+xi] = CL[yi][xi]
```

All threads cooperatively load AS and BS in different parallel patterns

Barrier inserted automatically by compiler

图7展示了此优化的影响。我们将*memory scopes*概念引入调度空间，使得计算阶段（代码中的AS和BS）可被标记为共享。在没有显式内存作用域的情况下，自动作用域推断会将计算阶段标记为线程本地。共享任务必须计算组内所有工作线程的依赖关系。此外，必须正确插入内存同步屏障，以确保共享加载的数据对消费者可见。最后，内存作用域不仅对GPU有用，还允许我们标记特殊内存缓冲区，并在针对专用DL加速器时创建特殊的底层代码生成规则。

4.3 张量化

深度学习工作负载具有高算术强度，通常可分解为张量运算符，如矩阵-矩阵乘法或一维卷积。这种自然分解催生了近期添加张量计算原语[1, 12, 21]的趋势。这些新原语为基于调度的编译带来了机遇与挑战：虽然使用它们能提升性能，但编译框架必须无缝集成这些原语。我们将此称为*tensorization*：它类似于SIMD架构的向量化，但存在显著差异。指令输入是多维的，具有固定或可变长度，且各自采用不同的数据布局。更重要的是，我们无法支持固定的原语集合，因为新兴加速器都配备各自变体的张量指令。因此我们需要一种*extensible*解决方案。

我们通过将目标硬件内联与调度分离，并引入张量内联声明机制，使张量化变得可扩展。我们使用相同的张量表达式语言来声明每个新硬件内联的行为及其相关的降级规则。以下代码展示了如何声明一个 8×8 张量硬件内联。

```
w, x = t.placeholder((8, 8)), t.placeholder((8, 8))
k = t.reduce_axis((0, 8))
y = t.compute((8, 8), lambda i, j:
    t.sum(w[i, k] * x[j, k], axis=k))

def gemm_intrin_lower(inputs, outputs):
    ww_ptr = inputs[0].access_ptr("r")
    xx_ptr = inputs[1].access_ptr("r")
    zz_ptr = outputs[0].access_ptr("w")
    compute = t.hardware_intrin("gemm8x8", ww_ptr, xx_ptr, zz_ptr)
    reset = t.hardware_intrin("fill_zero", zz_ptr)
    update = t.hardware_intrin("fuse_gemm8x8_add", ww_ptr, xx_ptr, zz_ptr)
    return compute, reset, update

gemm8x8 = t.decl_tensor_intrin(y.op, gemm_intrin_lower)
```

declare behavior

lowering rule to generate hardware intrinsics to carry out the computation

此外，我们引入了一个*tensorize*调度原语，用于将计算单元替换为相应的硬件内置函数。编译器将计算模式与硬件声明进行匹配，并将其降级为对应的硬件内置指令。

张量化将调度与特定硬件原语解耦，使得扩展TVM以支持新硬件架构变得容易。张量化调度生成的代码符合高性能计算的实践：将复杂操作分解为一系列微内核调用。我们还可以利用*tensorize*原语来使用手工优化的微内核，这在某些平台上可能带来益处。例如，我们通过利用位串行矩阵向量乘法微内核，为移动CPU实现了超低精度算子，这些算子可处理一比特或两比特宽的数据类型。该微内核将结果累加到逐渐增大的数据类型中，以最小化内存占用。将微内核作为张量内禀呈现给TVM，相比非张量化版本可实现高达 $1.5\times$ 的加速。

4.4 显式内存延迟隐藏

Latency hiding 指的是将内存操作与计算重叠进行，以最大化内存和计算资源的利用率。根据目标硬件后端的差异，需要采取不同的策略。在CPU上，内存延迟隐藏是通过同时多线程[14]或硬件预取[10, 20]隐式实现的。GPU依赖于多个线程束的快速上下文切换[44]。相比之下，诸如TPU[21]之类的专用深度学习加速器通常倾向于采用更精简的*decoupled access-execute* (DAE)架构[35]进行控制，并将细粒度同步的问题卸载给软件处理。

图9展示了一个降低运行时延迟的DAE硬件流水线。与单片硬件设计相比，该流水线能隐藏大部分内存访问开销，并几乎完全利用计算资源。为实现更高利用率，指令流必须通过细粒度同步操作进行增强。若缺少这些操作，依赖关系无法强制执行，将导致错误执行。因此，DAE硬件流水线需要在流水线阶段之间采用细粒度的依赖入队/出队操作来确保

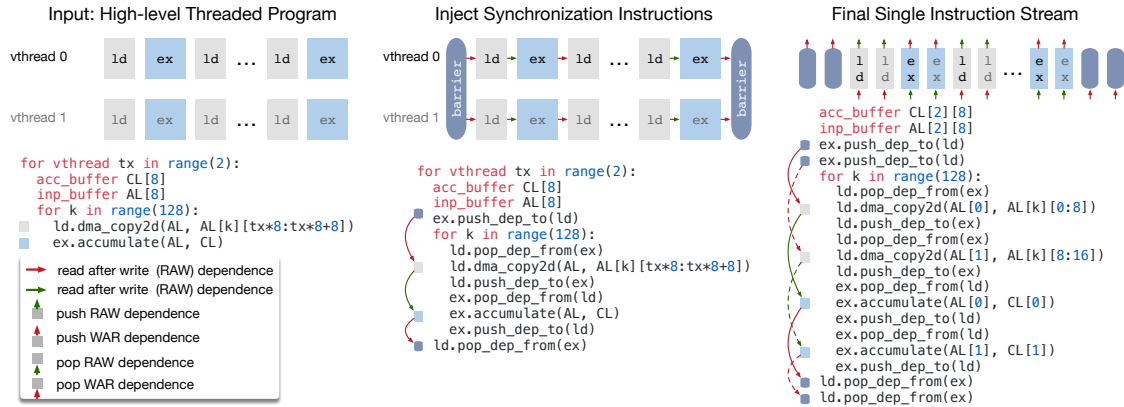


Figure 8: TVM virtual thread lowering transforms a virtual thread-parallel program to a single instruction stream; the stream contains explicit low-level synchronizations that the hardware can interpret to recover the pipeline parallelism required to hide memory access latency.

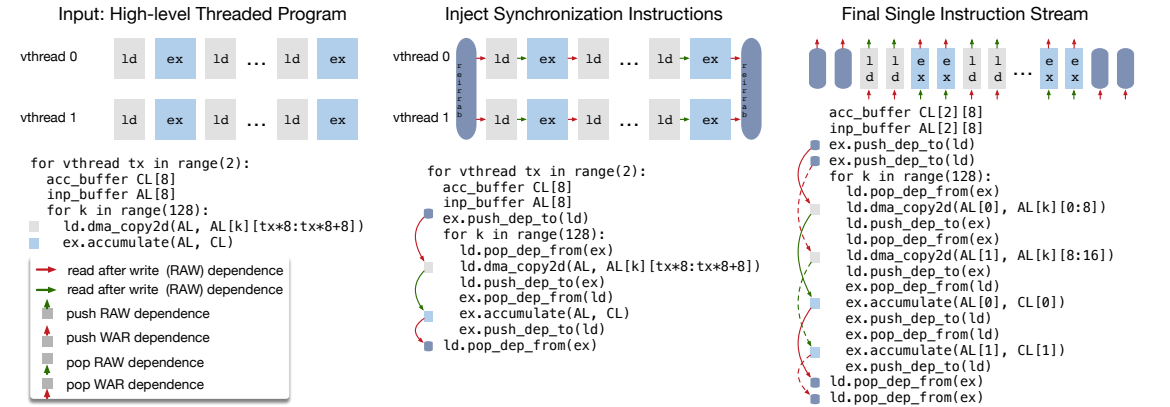


图8: TVM虚拟线程降低将虚拟线程并行程序转换为单一指令流; 该流包含硬件可解释的显式低级同步, 以恢复隐藏内存访问延迟所需的流水线并行性。

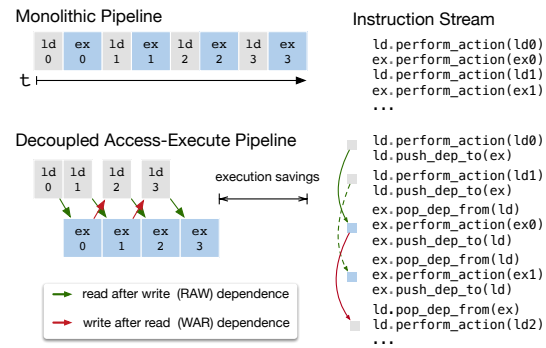


Figure 9: Decoupled Access-Execute in hardware hides most memory access latency by allowing memory and computation to overlap. Execution correctness is enforced by low-level synchronization in the form of dependence token enqueueing/dequeueing actions, which the compiler stack must insert in the instruction stream.

antee correct execution, as shown in Figure 9's instruction stream.

Programming DAE accelerators that require explicit low-level synchronization is difficult. To reduce the programming burden, we introduce a virtual threading scheduling primitive that lets programmers specify a high-level data parallel program as they would a hardware back-end with support for multithreading. TVM then automatically lowers the program to a single instruction stream with low-level explicit synchronization, as shown in Figure 8. The algorithm starts with a high-level multi-threaded program schedule and then inserts the necessary low-level synchronization operations to guarantee correct execution within each thread. Next, it interleaves operations of all virtual threads into a single instruction stream. Finally, the hardware recovers the

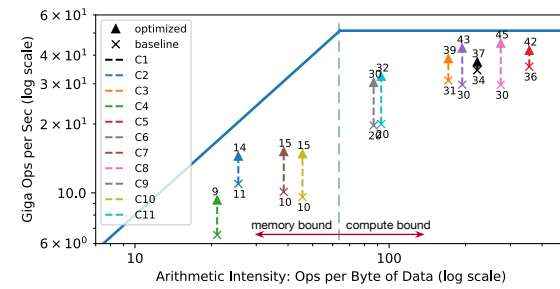


Figure 10: Roofline [47] of an FPGA-based DL accelerator running ResNet inference. With latency hiding enabled by TVM, performance of the benchmarks is brought closer to the roofline, demonstrating higher compute and memory bandwidth efficiency.

available pipeline parallelism dictated by the low-level synchronizations in the instruction stream.

Hardware Evaluation of Latency Hiding. We now demonstrate the effectiveness of latency hiding on a custom FPGA-based accelerator design, which we describe in depth in subsection 6.4. We ran each layer of ResNet on the accelerator and used TVM to generate two schedules: one with latency hiding, and one without. The schedule with latency hiding parallelized the program with virtual threads to expose pipeline parallelism and therefore hide memory access latency. Results are shown in Figure 10 as a roofline diagram [47]; roofline performance diagrams provide insight into how well a given system uses computation and memory resources for different benchmarks. Overall, latency hiding improved performance on all ResNet layers. Peak compute utilization increased from 70% with no latency hiding to 88% with latency hiding.

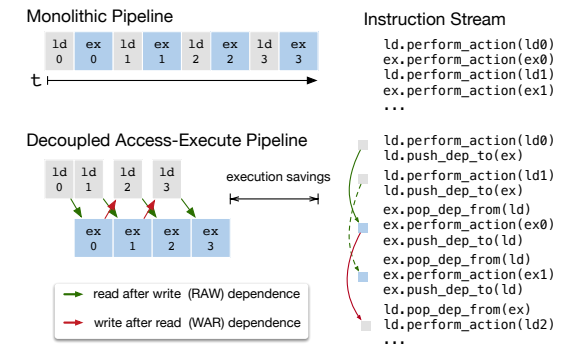


图9: 硬件中的解耦访问执行通过允许内存访问与计算重叠, 隐藏了大部分内存访问延迟。执行正确性通过依赖令牌入队/出队形式的低级同步来保证, 编译器栈必须在指令流中插入这些操作。

保证正确执行, 如图9的指令流所示。

编程需要显式低级同步的DAE加速器是困难的。为了减轻编程负担, 我们引入了一种虚拟线程调度原语, 让程序员能够像指定支持多线程的硬件后端那样, 指定高级数据并行程序。然后, TVM自动将程序降级为带有低级显式同步的单一指令流, 如图8所示。该算法从高级多线程程序调度开始, 然后插入必要的低级同步操作, 以确保每个线程内的正确执行。接着, 它将所有虚拟线程的操作交错排列成单一指令流。最后, 硬件恢复

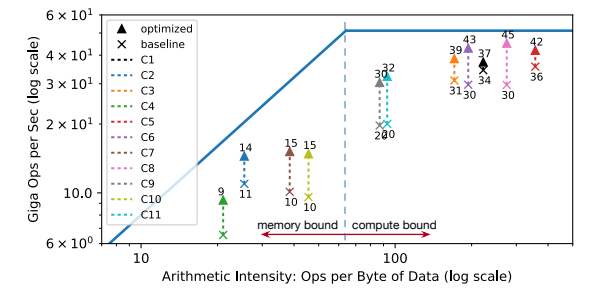


图10: 基于FPGA的深度学习加速器运行ResNet推理的Roofline模型[47]。通过TVM实现的延迟隐藏, 基准测试的性能更接近屋顶线, 展示了更高的计算和内存带宽效率。

可用流水线并行性由指令流中的低级同步决定。

延迟隐藏的硬件评估。我们现在在一个基于FPGA的定制加速器设计上展示延迟隐藏的有效性, 该设计在6.4小节中有详细描述。我们在加速器上运行了ResNet的每一层, 并使用TVM生成了两个调度方案: 一个带有延迟隐藏, 另一个没有。带有延迟隐藏的调度方案通过虚拟线程并行化程序, 以暴露流水线并行性, 从而隐藏内存访问延迟。结果如图10中的屋顶线图所示[47]; 屋顶线性能图揭示了给定系统在不同基准测试中如何有效利用计算和内存资源。总体而言, 延迟隐藏提高了所有ResNet层的性能。峰值计算利用率从无延迟隐藏时的70%提升至延迟隐藏时的88%。

5 Automating Optimization

Given the rich set of schedule primitives, our remaining problem is to find optimal operator implementations for each layer of a DL model. Here, TVM creates a specialized operator for the specific input shape and layout associated with each layer. Such specialization offers significant performance benefits (in contrast to handcrafted code that would target a smaller diversity of shapes and layouts), but it also raises automation challenges. The system needs to choose the schedule optimizations – such as modifying the loop order or optimizing for the memory hierarchy – as well as schedule-specific parameters, such as the tiling size and the loop unrolling factor. Such combinatorial choices create a large search space of operator implementations for each hardware back-end. To address this challenge, we built an *automated schedule optimizer* with two main components: a schedule explorer that *proposes* promising new configurations, and a machine learning cost model that *predicts* the performance of a given configuration. This section describes these components and TVM’s automated optimization flow (Figure 11).

5.1 Schedule Space Specification

We built a *schedule template specification API* to let a developer declare knobs in the schedule space. The template specification allows incorporation of a developer’s domain-specific knowledge, as necessary, when specifying possible schedules. We also created a *generic master template for each hardware back-end* that automatically extracts possible knobs based on the computation description expressed using the tensor expression language. At a high level, we would like to consider as many configurations as possible and let the optimizer manage the selection burden. Consequently, the optimizer must search over *billions* of possible configurations for the real world DL workloads used in our experiments.

5.2 ML-Based Cost Model

One way to find the best schedule from a large configuration space is through blackbox optimization, i.e., auto-tuning. This method is used to tune high performance computing libraries [15, 46]. However, auto-tuning requires many experiments to identify a good configuration.

An alternate approach is to build a predefined cost model to guide the search for a particular hardware back-end instead of running all possibilities and measuring their performance. Ideally, a perfect cost model considers all factors affecting performance: memory access patterns, data reuse, pipeline dependencies, and thread-

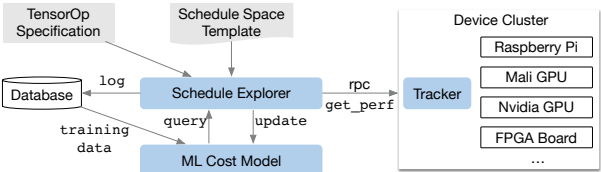


Figure 11: Overview of automated optimization framework. A schedule explorer examines the schedule space using an ML-based cost model and chooses experiments to run on a distributed device cluster via RPC. To improve its predictive power, the ML model is updated periodically using collected data recorded in a database.

Method Category	Data Cost	Model Bias	Need Hardware Info	Learn from History
Blackbox auto-tuning	high	none	no	no
Predefined cost model	none	high	yes	no
ML based cost model	low	low	no	yes

Table 1: Comparison of automation methods. Model bias refers to inaccuracy due to modeling.

ing patterns, among others. This approach, unfortunately, is burdensome due to the increasing complexity of modern hardware. Furthermore, every new hardware target requires a new (predefined) cost model.

We instead take a statistical approach to solve the cost modeling problem. In this approach, a schedule explorer proposes configurations that may improve an operator’s performance. For each schedule configuration, we use an ML model that takes the lowered loop program as input and predicts its running time on a given hardware back-end. The model, trained using runtime measurement data collected during exploration, does not require the user to input detailed hardware information. We update the model periodically as we explore more configurations during optimization, which improves accuracy for other related workloads, as well. In this way, the quality of the ML model improves with more experimental trials. Table 1 summarizes the key differences between automation methods. ML-based cost models strike a balance between auto-tuning and predefined cost modeling and can benefit from the historical performance data of related workloads.

Machine Learning Model Design Choices. We must consider two key factors when choosing which ML model the schedule explorer will use: *quality* and *speed*. The schedule explorer queries the cost model frequently, which incurs overheads due to model prediction time and model refitting time. To be useful, these overheads must be smaller than the time it takes to measure per-

5 自动化优化

鉴于丰富的调度原语集合，我们剩余的问题是深度学习模型的每一层寻找最优的算子实现。为此，TVM针对每层特定的输入形状和布局创建了专门的算子。这种专门化带来了显著的性能优势（相较于针对较少形状和布局多样性编写的手工代码），但也带来了自动化挑战。系统需要选择调度优化——例如修改循环顺序或针对内存层次结构进行优化——以及调度特定参数，如分块大小和循环展开因子。这些组合选择为每个硬件后端创建了一个庞大的算子实现搜索空间。为应对这一挑战，我们构建了一个包含两个主要组件的*automated schedule optimizer*：一个调度探索器，用于*proposes*有前景的新配置；以及一个机器学习成本模型，用于*predicts*给定配置的性能。本节将描述这些组件及TVM的自动化优化流程（图11）。

5.1 进度空间规范

我们构建了一个*schedule template specification API*，让开发者能够在调度空间中声明旋钮。模板规范允许在指定可能的调度时，根据需要融入开发者的领域特定知识。我们还创建了一个*generic master template for each hardware back-end*，能够基于使用张量表达式语言描述的计算，自动提取可能的旋钮。从高层次来看，我们希望考虑尽可能多的配置，并让优化器承担选择的重担。因此，优化器必须在我们实验所用的真实世界深度学习工作负载中，搜索*billions*个可能的配置。

5.2 基于机器学习的成本模型

在庞大的配置空间中寻找最优调度的一种方法是通过黑盒优化，即自动调优。这种方法常用于调优高性能计算库[15, 46]。然而，自动调优需要进行大量实验才能确定一个良好的配置。

另一种方法是构建一个预定义的成本模型来指导针对特定硬件后端的搜索，而不是运行所有可能性并测量其性能。理想情况下，一个完美的成本模型会考虑所有影响性能的因素：内存访问模式、数据重用、流水线依赖关系以及线程

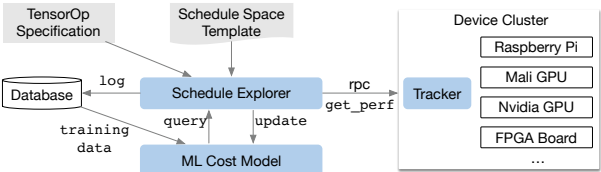


图11：自动化优化框架概览。调度探索器利用基于机器学习的成本模型检查调度空间，并通过RPC选择在分布式设备集群上运行的实验。为提升预测能力，机器学习模型会定期使用数据库中记录的收集数据进行更新。

Method Category	Data Cost	Model Bias	Need Hardware Info	Learn from History
Blackbox auto-tuning	high	none	no	no
Predefined cost model	none	high	yes	no
ML based cost model	low	low	no	yes

表1：自动化方法比较。模型偏差指因建模导致的不准确性。

模式等。这种方法，不幸的是，由于现代硬件的日益复杂而变得繁重。此外，每个新的硬件目标都需要一个新的（预定义的）成本模型。

我们转而采用统计方法来解决成本建模问题。该方法中，调度探索器会提出可能提升算子性能的配置方案。针对每个调度配置，我们使用一个机器学习模型：该模型以降级后的循环程序为输入，预测其在给定硬件后端上的运行时间。该模型利用探索过程中收集的运行时测量数据进行训练，无需用户输入详细的硬件信息。我们在优化过程中探索更多配置时会定期更新模型，这同时提升了其他相关工作负载的预测准确性。通过这种方式，机器学习模型的质量随着实验次数的增加而不断提高。表1总结了各类自动化方法的核心差异。基于机器学习的成本模型在自动调优与预定义成本建模之间取得了平衡，并能从相关工作负载的历史性能数据中持续获益。

机器学习模型设计选择。在选择调度探索器将使用哪种机器学习模型时，我们必须考虑两个关键因素：*quality*和*speed*。调度探索器频繁查询成本模型，这会因模型预测时间和模型重新拟合时间而产生开销。为了实用，这些开销必须小于测量每个-

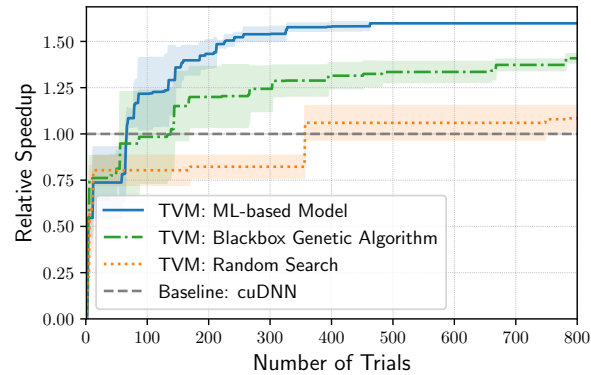


Figure 12: Comparison of different automation methods for a conv2d operator in ResNet-18 on TITAN X. The ML-based model starts with no training data and uses the collected data to improve itself. The Y-axis is the speedup relative to cuDNN. We observe a similar trend for other workloads.

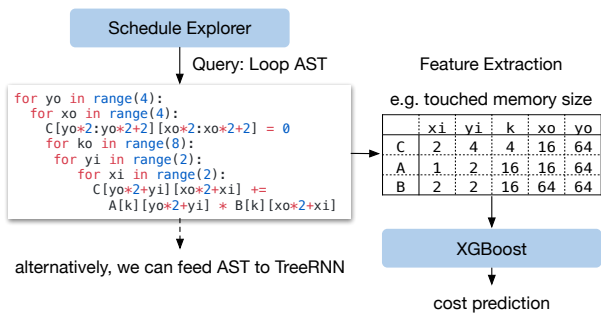


Figure 13: Example workflow of ML cost models. XGBoost predicts costs based on loop program features. TreeRNN directly summarizes the AST.

formance on real hardware, which can be on the order of seconds depending on the specific workload/hardware target. This speed requirement differentiates our problem from traditional hyperparameter tuning problems, where the cost of performing measurements is very high relative to model overheads, and more expensive models can be used. In addition to the choice of model, we need to choose an objective function to train the model, such as the error in a configuration’s predicted running time. However, since the explorer selects the top candidates based only on the relative order of the prediction (A runs faster than B), we need not predict the absolute execution times directly. Instead, we use a rank objective to predict the relative order of runtime costs.

We implement several types of models in our ML optimizer. We employ a *gradient tree boosting model* (based on XGBoost [8]), which makes predictions based on features extracted from the loop program; these features in-

clude the memory access count and reuse ratio of each memory buffer at each loop level, as well as a one-hot encoding of loop annotations such as “vectorize”, “unroll”, and “parallel.” We also evaluate a *neural network model* that uses TreeRNN [38] to summarize the loop program’s AST without feature engineering. Figure 13 summarizes the workflow of the cost models. We found that tree boosting and TreeRNN have similar predictive quality. However, the former performs prediction twice as fast and costs much less time to train. As a result, we chose gradient tree boosting as the default cost model in our experiments. Nevertheless, we believe that both approaches are valuable and expect more future research on this problem.

On average, the tree boosting model does prediction in 0.67 ms, thousands of times faster than running a real measurement. Figure 12 compares an ML-based optimizer to blackbox auto-tuning methods; the former finds better configurations much faster than the latter.

5.3 Schedule Exploration

Once we choose a cost model, we can use it to select promising configurations on which to iteratively run real measurements. In each iteration, the explorer uses the ML model’s predictions to select a batch of candidates on which to run the measurements. The collected data is then used as training data to update the model. If no initial training data exists, the explorer picks random candidates to measure.

The simplest exploration algorithm enumerates and runs every configuration through the cost model, selecting the top- k predicted performers. However, this strategy becomes intractable with large search spaces. Instead, we run a parallel simulated annealing algorithm [22]. The explorer starts with random configurations, and, at each step, randomly walks to a nearby configuration. This transition is successful if cost decreases as predicted by the cost model. It is likely to fail (reject) if the target configuration has a higher cost. The random walk tends to converge on configurations that have lower costs as predicted by the cost model. Exploration states persist across cost model updates; we continue from the last configuration after these updates.

5.4 Distributed Device Pool and RPC

A *distributed device pool* scales up the running of on-hardware trials and enables fine-grained resource sharing among multiple optimization jobs. TVM implements a customized, RPC-based distributed device pool that enables clients to run programs on a specific type of device. We can use this interface to compile a program on the host compiler, request a remote device, run the

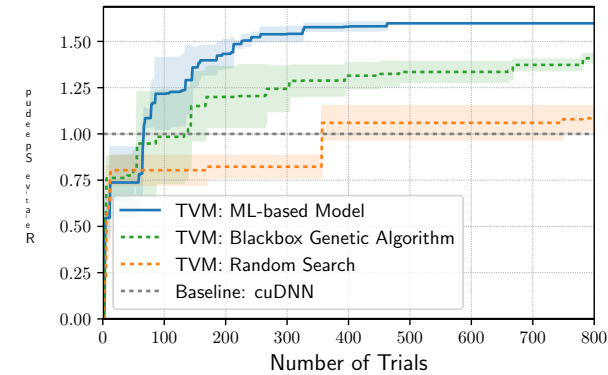


图12: 在TITAN X上对ResNet-18中conv2d算子不同自动化方法的比较。基于机器学习的方法从零训练数据开始, 并利用收集的数据进行自我优化。Y轴表示相对于cuDNN的加速比。我们观察到其他工作负载也呈现相似趋势。

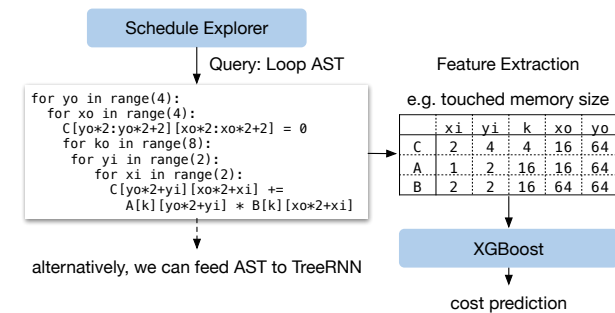


图13: 机器学习成本模型的工作流程示例。XGBoost基于循环程序特征预测成本, TreeRNN则直接对抽象语法树进行归纳。

在实际硬件上的性能, 根据具体的工作负载/硬件目标, 可能达到秒级。这一速度要求将我们的问题与传统超参数调优问题区分开来, 后者执行测量的成本相对于模型开销非常高, 且可以使用更昂贵的模型。除了模型的选择, 我们还需要选择一个目标函数来训练模型, 例如配置预测运行时间的误差。然而, 由于探索器仅根据预测的相对顺序 (A比B运行更快) 来选择最佳候选, 我们无需直接预测绝对执行时间。相反, 我们使用排序目标来预测运行时成本的相对顺序。

我们在机器学习优化器中实现了多种类型的模型。我们采用了基于XGBoost [8]的 *gradient tree boosting model* (), 该模型根据从循环程序中提取的特征进行预测; 这些特征包

包括每个循环级别上每个内存缓冲区的内存访问次数和重用率, 以及循环注释 (如“向量化”、“展开”和“并行”) 的独热编码。我们还评估了一种使用TreeRNN [38]来总结循环程序AST而无需特征工程的 *neural network model*。图13总结了成本模型的工作流程。我们发现树提升和TreeRNN具有相似的预测质量。然而, 前者的预测速度快一倍, 且训练时间少得多。因此, 我们在实验中选择了梯度树提升作为默认成本模型。尽管如此, 我们认为这两种方法都有价值, 并期待未来在这一问题上有更多研究。

平均而言, 树提升模型的预测时间为0.67毫秒, 比实际测量快数千倍。图12将基于机器学习的优化器与黑盒自动调优方法进行了对比; 前者比后者更快地找到了更优配置。

5.3 进度探索

一旦我们选定了一个成本模型, 就可以用它来选择有前景的配置, 并在此基础上迭代运行实际测量。在每次迭代中, 探索器利用机器学习模型的预测来挑选一批候选配置进行测量。收集到的数据随后被用作训练数据来更新模型。如果初始训练数据不存在, 探索器会随机选择候选配置进行测量。

最简单的探索算法会枚举并通过成本模型运行每种配置, 选择预测性能前 k 的配置。然而, 对于大型搜索空间, 这种策略变得难以处理。因此, 我们采用并行模拟退火算法[22]。探索器从随机配置开始, 在每一步中随机游走到附近的配置。如果成本模型预测成本降低, 则此转换成功。如果目标配置成本更高, 则很可能失败 (拒绝)。随机游走倾向于收敛于成本模型预测成本较低的配置。探索状态在成本模型更新后保持不变; 我们在这些更新后从最后一个配置继续。

5.4 分布式设备池与RPC

*distributed device pool*扩展了硬件上试验的运行规模, 并实现了多个优化任务间的细粒度资源共享。TVM实现了一个定制的、基于RPC的分布式设备池, 使客户端能够在特定类型的设备上运行程序。我们可以通过此接口在主机编译器上编译程序, 请求远程设备并运行

Name	Operator	H, W	IC, OC	K, S
C1	conv2d	224, 224	3, 64	7, 2
C2	conv2d	56, 56	64, 64	3, 1
C3	conv2d	56, 56	64, 64	1, 1
C4	conv2d	56, 56	64, 128	3, 2
C5	conv2d	56, 56	64, 128	1, 2
C6	conv2d	28, 28	128, 128	3, 1
C7	conv2d	28, 28	128, 256	3, 2
C8	conv2d	28, 28	128, 256	1, 2
C9	conv2d	14, 14	256, 256	3, 1
C10	conv2d	14, 14	256, 512	3, 2
C11	conv2d	14, 14	256, 512	1, 2
C12	conv2d	7, 7	512, 512	3, 1

Name	Operator	H, W	IC	K, S
D1	depthwise conv2d	112, 112	32	3, 1
D2	depthwise conv2d	112, 112	64	3, 2
D3	depthwise conv2d	56, 56	128	3, 1
D4	depthwise conv2d	56, 56	128	3, 2
D5	depthwise conv2d	28, 28	256	3, 1
D6	depthwise conv2d	28, 28	256	3, 2
D7	depthwise conv2d	14, 14	512	3, 1
D8	depthwise conv2d	14, 14	512	3, 2
D9	depthwise conv2d	7, 7	1024	3, 1

Table 2: Configurations of all conv2d operators in ResNet-18 and all depthwise conv2d operators in MobileNet used in the single kernel experiments. H/W denotes height and width, IC input channels, OC output channels, K kernel size, and S stride size. All ops use “SAME” padding. All depthwise conv2d operations have channel multipliers of 1.

function remotely, and access results in the same script on the host. TVM’s RPC supports dynamic upload and runs cross-compiled modules and functions that use its runtime convention. As a result, the same infrastructure can perform a single workload optimization and end-to-end graph inference. Our approach automates the compile, run, and profile steps across multiple devices. This infrastructure is especially critical for embedded devices, which traditionally require tedious manual effort for cross-compilation, code deployment, and measurement.

6 Evaluation

TVM’s core is implemented in C++ (~50k LoC). We provide language bindings to Python and Java. Earlier sections of this paper evaluated the impact of several individual optimizations and components of TVM, namely, *operator fusion* in Figure 4, *latency hiding* in Figure 10, and the *ML-based cost model* in Figure 12. We now focus on an end-to-end evaluation that aims to answer the following questions:

- Can TVM optimize DL workloads over multiple platforms?
- How does TVM compare to existing DL frame-

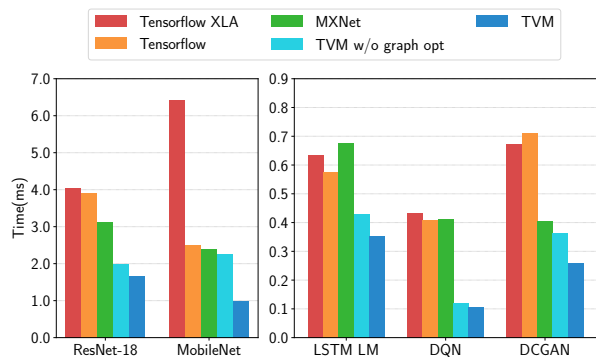


Figure 14: GPU end-to-end evaluation for TVM, MXNet, Tensorflow, and Tensorflow XLA. Tested on the NVIDIA Titan X.

works (which rely on heavily optimized libraries) on each back-end?

- Can TVM support new, emerging DL workloads (e.g., depthwise convolution, low precision operations)?
- Can TVM support and optimize for new specialized accelerators?

To answer these questions, we evaluated TVM on four types of platforms: (1) a server-class GPU, (2) an embedded GPU, (3) an embedded CPU, and (4) a DL accelerator implemented on a low-power FPGA SoC. The benchmarks are based on real world DL inference workloads, including ResNet [16], MobileNet [19], the LSTM Language Model [48], the Deep Q Network (DQN) [28] and Deep Convolutional Generative Adversarial Networks (DCGAN) [31]. We compare our approach to existing DL frameworks, including MxNet [9] and TensorFlow [2], that rely on highly engineered, vendor-specific libraries. TVM performs end-to-end automatic optimization and code generation *without the need for an external operator library*.

6.1 Server-Class GPU Evaluation

We first compared the end-to-end performance of deep neural networks TVM, MXNet (v1.1), Tensorflow (v1.7), and Tensorflow XLA on an Nvidia Titan X. MXNet and Tensorflow both use cuDNN v7 for convolution operators; they implement their own versions of depthwise convolution since it is relatively new and not yet supported by the latest libraries. They also use cuBLAS v8 for matrix multiplications. On the other hand, Tensorflow XLA uses JIT compilation.

Figure 14 shows that TVM outperforms the baselines, with speedups ranging from $1.6\times$ to $3.8\times$ due to both joint graph optimization and the automatic optimizer, which generates high-performance fused opera-

Name	Operator	H, W	IC, OC	K, S
C1	conv2d	224, 224	3, 64	7, 2
C2	conv2d	56, 56	64, 64	3, 1
C3	conv2d	56, 56	64, 64	1, 1
C4	conv2d	56, 56	64, 128	3, 2
C5	conv2d	56, 56	64, 128	1, 2
C6	conv2d	28, 28	128, 128	3, 1
C7	conv2d	28, 28	128, 256	3, 2
C8	conv2d	28, 28	128, 256	1, 2
C9	conv2d	14, 14	256, 256	3, 1
C10	conv2d	14, 14	256, 512	3, 2
C11	conv2d	14, 14	256, 512	1, 2
C12	conv2d	7, 7	512, 512	3, 1

Name	Operator	H, W	IC	K, S
D1	depthwise conv2d	112, 112	32	3, 1
D2	depthwise conv2d	112, 112	64	3, 2
D3	depthwise conv2d	56, 56	128	3, 1
D4	depthwise conv2d	56, 56	128	3, 2
D5	depthwise conv2d	28, 28	256	3, 1
D6	depthwise conv2d	28, 28	256	3, 2
D7	depthwise conv2d	14, 14	512	3, 1
D8	depthwise conv2d	14, 14	512	3, 2
D9	depthwise conv2d	7, 7	1024	3, 1

表2：单核实验中使用的ResNet-18所有conv2d算子及MobileNet所有深度可分离conv2d算子的配置。 H/W 表示高度和宽度， IC 为输入通道数， OC 为输出通道数， K 为卷积核尺寸， S 为步长。所有算子均采用“SAME”填充方式。所有深度可分离conv2d运算的通道乘数均为1。

远程运行功能，并在主机上的同一脚本中访问结果。TVM的RPC支持动态上传，并运行使用其运行时约定的交叉编译模块和函数。因此，同一基础设施能够执行单一工作负载优化和端到端的图推理。我们的方法自动化了跨多个设备的编译、运行和分析步骤。这一基础设施对于嵌入式设备尤为关键，传统上这些设备需要繁琐的手动操作来进行交叉编译、代码部署和性能测量。

6 评估

TVM的核心由C语言实现，代码量达5万行。我们提供了Python和Java的语言绑定接口。本文前几节评估了TVM中多项独立优化与组件的效果，即图4中的算子融合、图10中的张量运算优化以及图12中的内存延迟隐藏技术。接下来我们将聚焦端到端的整体评估，旨在回答以下问题：

- TVM能否在多个平台上优化深度学习工作负载？
- TVM与现有深度学习框架相比如何

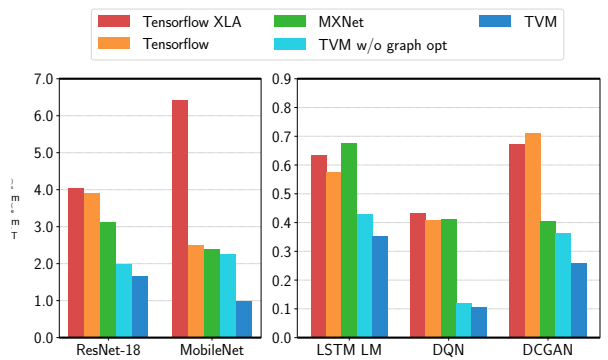


图14：TVM、MXNet、TensorFlow及TensorFlow XLA的GPU端到端评估。测试基于NVIDIA Titan X进行。

每个后端上的作品（依赖于高度优化的库）？

- TVM能否支持新兴的深度学习工作负载（例如深度可分离卷积、低精度运算）？
- TVM能否支持并优化新型专用加速器？

为了回答这些问题，我们在四类平台上评估了TVM：(1) 服务器级GPU，(2) 嵌入式GPU，(3) 嵌入式CPU，以及(4) 在低功耗FPGA SoC上实现的深度学习加速器。基准测试基于真实的深度学习推理工作负载，包括ResNet [16]、MobileNet [19]、LSTM语言模型 [48]、深度Q网络（DQN）[28]以及深度卷积生成对抗网络（DCGAN）[31]。我们将我们的方法与现有的深度学习框架进行了比较，包括依赖高度工程化、厂商特定库的MxNet [9]和TensorFlow [2]。TVM执行端到端的自动优化和代码生成 *without the need for an external operator library*。

6.1 服务器级GPU评估

我们首先在Nvidia Titan X上比较了深度神经网络TVM、MXNet (v1.1)、Tensorflow (v1.7)和Tensorflow XLA的端到端性能。MXNet和Tensorflow均使用cuDNN v7处理卷积算子；由于深度卷积相对较新且最新库尚未支持，它们各自实现了该运算的版本。同时，这两者还采用cuBLAS v8进行矩阵乘法运算。而Tensorflow XLA则使用即时编译技术。

图14显示，TVM优于基线，由于联合图优化和自动优化器（能生成高性能融合操作）的共同作用，其加速比范围从 $1.6\times$ 到 $3.8\times$ 。

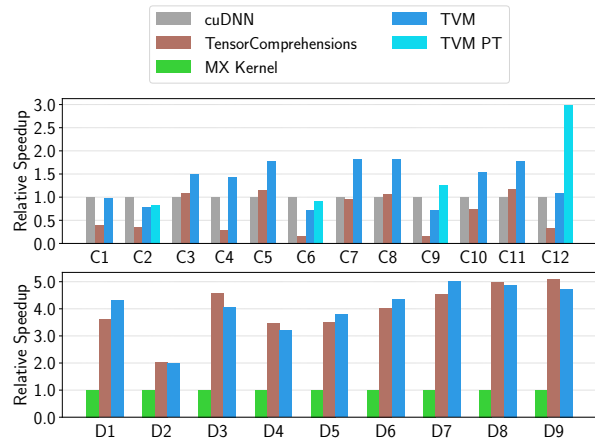


Figure 15: Relative speedup of all conv2d operators in ResNet-18 and all depthwise conv2d operators in MobileNet. Tested on a TITAN X. See Table 2 for operator configurations. We also include a weight pre-transformed Winograd [25] for 3x3 conv2d (TVM PT).

tors. DQN’s 3.8 x speedup results from its use of unconventional operators (4×4 conv2d, $\text{strides}=2$) that are not well optimized by cuDNN; the ResNet workloads are more conventional. TVM automatically finds optimized operators in both cases.

To evaluate the effectiveness of operator level optimization, we also perform a breakdown comparison for each tensor operator in ResNet and MobileNet, shown in Figure 15. We include TensorComprehension (TC, commit: ef644ba) [42], a recently introduced auto-tuning framework, as an additional baseline.² TC results include the best kernels it found in $10 \text{ generations} \times 100 \text{ population} \times 2 \text{ random seeds}$ for each operator (i.e., 2000 trials per operator). 2D convolution, one of the most important DL operators, is heavily optimized by cuDNN. However, TVM can still generate better GPU kernels for most layers. Depthwise convolution is a newly introduced operator with a simpler structure [19]. In this case, both TVM and TC can find fast kernels compared to MXNet’s handcrafted kernels. TVM’s improvements are mainly due to its exploration of a large schedule space and an effective ML-based search algorithm.

6.2 Embedded CPU Evaluation

We evaluated the performance of TVM on an ARM Cortex A53 (Quad Core 1.2GHz). We used Tensorflow Lite (TFLite, commit: 7558b085) as our baseline system. Figure 17 compares TVM operators to hand-optimized

²According to personal communication [41], TC is not yet meant to be used for compute-bound problems. However, it is still a good reference baseline to include in the comparison.

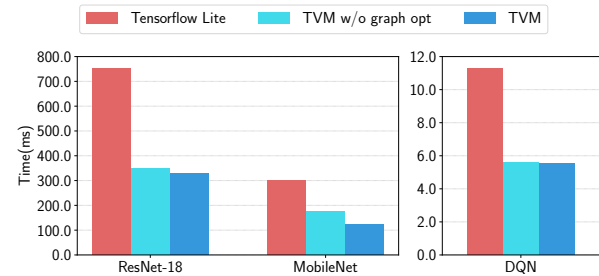


Figure 16: ARM A53 end-to-end evaluation of TVM and TFLite.

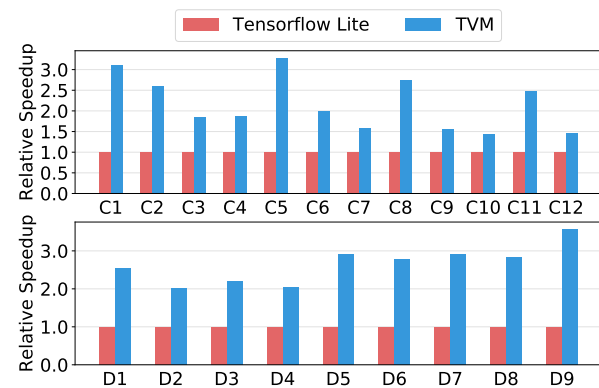


Figure 17: Relative speedup of all conv2d operators in ResNet-18 and all depthwise conv2d operators in MobileNet. Tested on ARM A53. See Table 2 for the configurations of these operators.

ones for ResNet and MobileNet. We observe that TVM generates operators that outperform the hand-optimized TFLite versions for both neural network workloads. This result also demonstrates TVM’s ability to quickly optimize emerging tensor operators, such as depthwise convolution operators. Finally, Figure 16 shows an end-to-end comparison of three workloads, where TVM outperforms the TFLite baseline.³

Ultra Low-Precision Operators We demonstrate TVM’s ability to support ultra low-precision inference [13, 33] by generating highly optimized operators for fixed-point data types of less than 8-bits. Low-precision networks replace expensive multiplication with vectorized bit-serial multiplication that is composed of bitwise *and* popcount reductions [39]. Achieving efficient low-precision inference requires packing quantized data types into wider standard data types, such as `int8` or `int32`. Our system generates code that outperforms hand-optimized libraries from Caffe2 (commit: 39e07f7)

³DCGAN and LSTM results are not presented because they are not yet supported by the baseline.

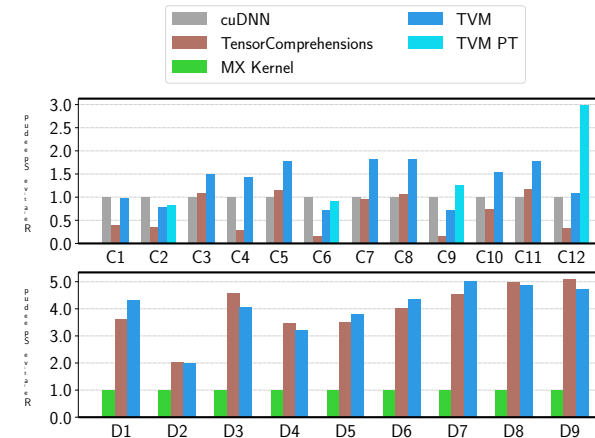


图15: ResNet-18中所有conv2d算子以及MobileNet中所有深度可分离conv2d算子的相对加速比。测试平台为TITAN X。算子配置详见表2。我们还为3x3 conv2d加入了权重预变换Winograd方法[25] (TVM PT)。

tors. DQN的3.8倍加速源于其使用了非常规算子 (4×4 conv2d, $\text{strides}=2$)，这些算子未得到cuDNN的良好优化；而ResNet的工作负载更为常规。TVM在这两种情况下均能自动找到优化后的算子。

为了评估算子级别优化的效果，我们还对ResNet和MobileNet中的每个张量算子进行了细分比较，如图15所示。我们引入了TensorComprehension (TC, 提交版本: ef644ba) [42]作为额外基准，这是一个近期提出的自动调优框架。² TC的结果包含了其在10代 $\times$ 100种群 $\times$ 2个随机种子条件下为每个算子找到的最佳内核（即每个算子进行2000次尝试）。二维卷积作为最重要的深度学习算子之一，已由cuDNN进行了深度优化。然而，TVM仍能为大多数层生成更优的GPU内核。深度可分离卷积是一种新引入的、结构更简洁的算子[19]。在此情况下，TVM和TC均能找到比MXNet手工优化内核更快的方案。TVM的改进主要得益于其对大规模调度空间的探索以及高效的基于机器学习的搜索算法。

6.2 嵌入式CPU评估

我们在ARM Cortex A53（四核1.2GHz）上评估了TVM的性能。我们使用TensorFlow Lite (TFLite, 提交版本: 7558b085) 作为基准系统。图17将TVM算子与手动优化的

²According to personal communication [41], TC is not yet meant to be used for compute-bound problems. However, it is still a good reference baseline to include in the comparison.

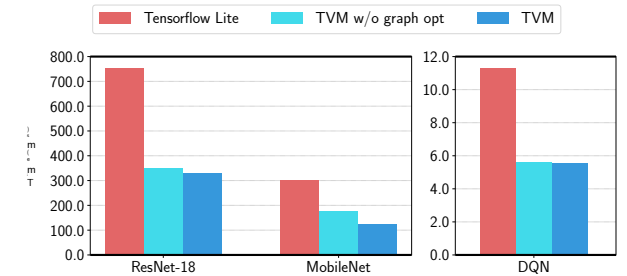


图16: TVM与TFLite在ARM A53平台上的端到端评估。

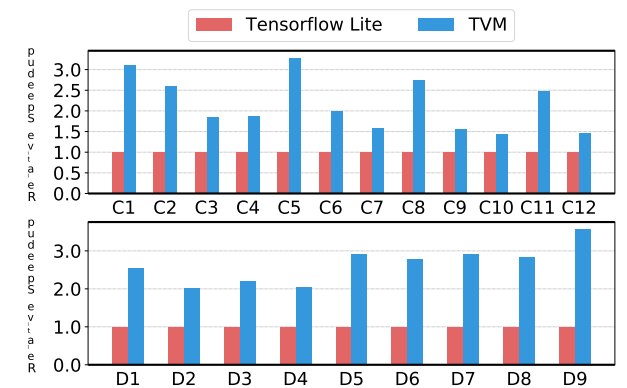


图17: ResNet-18中所有conv2d算子以及MobileNet中所有深度可分离卷积算子的相对加速比。测试平台为ARM A53。相关算子的配置详见表2。

针对ResNet和MobileNet的算子。我们观察到，TVM生成的算子在两种神经网络工作负载上均优于手动优化的TFLite版本。这一结果也证明了TVM快速优化新兴张量算子（如深度卷积算子）的能力。最后，图16展示了三种工作负载的端到端对比，其中TVM的表现超越了TFLite基线。³

超低精度算子 我们通过为小于8位的定点数据类型生成高度优化的算子，展示了TVM支持超低精度推理的能力[13, 33]。低精度网络用由按位*and*和人口计数归约[39]组成的向量化位串行乘法替代了昂贵的乘法运算。实现高效的低精度推理需要将量化数据类型打包到更宽的标准数据类型中，例如`int8`或`int32`。我们的系统生成的代码性能优于Caffe2（提交版本: 39e07f7）中手动优化的库。

³DCGAN and LSTM results are not presented because they are not yet supported by the baseline.

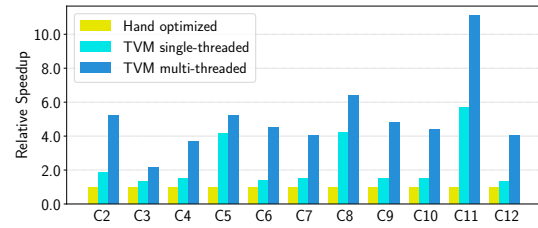


Figure 18: Relative speedup of single- and multi-threaded low-precision conv2d operators in ResNet. Baseline was a single-threaded, hand-optimized implementation from Caffe2 (commit: 39e07f7). C5, C3 are 1x1 convolutions that have less compute intensity, resulting in less speedup by multi-threading.

[39]. We implemented an ARM-specific *tensorization* intrinsic that leverages ARM instructions to build an efficient, low-precision matrix-vector microkernel. We then used TVM’s automated optimizer to explore the scheduling space.

Figure 18 compares TVM to the Caffe2 ultra low-precision library on ResNet for 2-bit activations, 1-bit weights inference. Since the baseline is single threaded, we also compare it to a single-threaded TVM version. Single-threaded TVM outperforms the baseline, particularly for C5, C8, and C11 layers; these are convolution layers of kernel size 1×1 and stride of 2 for which the ultra low-precision baseline library is not optimized. Furthermore, we take advantage of additional TVM capabilities to produce a parallel library implementation that shows improvement over the baseline. In addition to the 2-bit+1-bit configuration, TVM can generate and optimize for other precision configurations that are unsupported by the baseline library, offering improved flexibility.

6.3 Embedded GPU Evaluation

For our mobile GPU experiments, we ran our end-to-end pipeline on a Firefly-RK3399 board equipped with an ARM Mali-T860MP4 GPU. The baseline was a vendor-provided library, the ARM Compute Library (v18.03). As shown in Figure 19, we outperformed the baseline on three available models for both float16 and float32 (DCGAN and LSTM are not yet supported by the baseline). The speedup ranged from $1.2\times$ to $1.6\times$.

6.4 FPGA Accelerator Evaluation

Vanilla Deep Learning Accelerator We now relate how TVM tackled accelerator-specific code generation on a generic inference accelerator design we prototyped on an FPGA. We used in this evaluation the Vanilla Deep

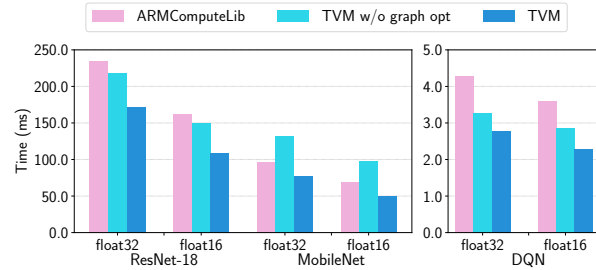


Figure 19: End-to-end experiment results on Mali-T860MP4. Two data types, float32 and float16, were evaluated.

Learning Accelerator (VDLA) – which distills characteristics from previous accelerator proposals [12, 21, 27] into a minimalist hardware architecture – to demonstrate TVM’s ability to generate highly efficient schedules that can target specialized accelerators. Figure 20 shows the high-level hardware organization of the VDLA architecture. VDLA is programmed as a tensor processor to efficiently execute operations with high compute intensity (e.g. matrix multiplication, high dimensional convolution). It can perform load/store operations to bring blocked 3-dimensional tensors from DRAM into a contiguous region of SRAM. It also provides specialized on-chip memories for network parameters, layer inputs (narrow data type), and layer outputs (wide data type). Finally, VDLA provides explicit synchronization control over successive loads, computes, and stores to maximize the overlap between memory and compute operations.

Methodology. We implemented the VDLA design on a low-power PYNQ board that incorporates an ARM Cortex A9 dual core CPU clocked at 667MHz and an Artix-7 based FPGA fabric. On these modest FPGA resources, we implemented a 16×16 matrix-vector unit clocked at 200MHz that performs products of 8-bit values and accumulates them into a 32-bit register every cycle. The theoretical peak throughput of this VDLA design is about 102.4GOPS/s. We allocated 32kB of resources for activation storage, 32kB for parameter storage, 32kB for microcode buffers, and 128kB for the register file. These on-chip buffers are by no means large enough to provide sufficient on-chip storage for a single layer of ResNet and therefore enable a case study on effective memory reuse and latency hiding.

We built a driver library for VDLA with a C runtime API that constructs instructions and pushes them to the target accelerator for execution. Our code generation algorithm then translates the accelerator program to a series of calls into the runtime API. Adding the specialized accelerator back-end took $\sim 2k$ LoC in Python.

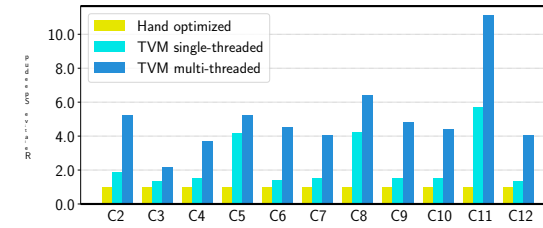


图18: ResNet中单线程与多线程低精度conv2d算子的相对加速比。基准为Caffe2（提交版本：39e07f7）的单线程手动优化实现。C5、C3是计算强度较低的1x1卷积，因此多线程带来的加速效果较小。

[39]。我们实现了一个ARM专用的*tensorization*内置函数，该函数利用ARM指令构建了一个高效、低精度的矩阵-向量微内核。随后，我们使用TVM的自动优化器来探索调度空间。

图18比较了TVM与Caffe2超低精度库在ResNet上针对2位激活、1位权重推理的性能。由于基准测试采用单线程，我们同时将其与单线程TVM版本进行对比。单线程TVM表现优于基准，尤其在C5、C8和C11层；这些是卷积核尺寸为 1×1 、步长为2的卷积层，而超低精度基准库未对此类配置进行优化。此外，我们利用TVM的额外功能生成了并行库实现，其性能较基准有所提升。除2位+1位配置外，TVM还能针对基准库不支持的其他精度配置进行生成和优化，从而提供更强的灵活性。

6.3 嵌入式GPU评估

在我们的移动GPU实验中，我们在配备ARM Mali-T860MP4 GPU的Firefly-RK3399开发板上运行了端到端流程。基准测试采用的是供应商提供的ARM Compute Library (v18.03)。如图19所示，我们在三个可用模型上（包括float16和float32精度）的表现均优于基准（DCGAN和LSTM尚未被基准库支持）。加速比范围从 $1.2\times$ 到 $1.6\times$ 。

6.4 FPGA加速器评估

Vanilla深度学习加速器我们现在介绍TVM如何在我们的FPGA上原型设计的通用推理加速器上处理特定于加速器的代码生成。在此次评估中，我们使用了Vanilla Deep

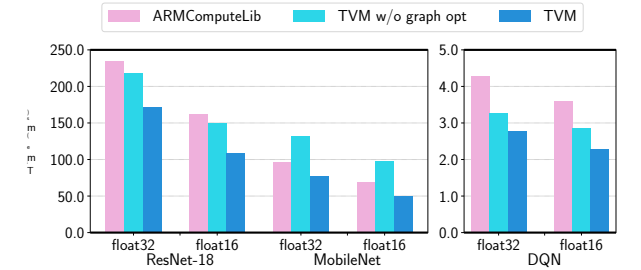


图19: Mali-T860MP4上的端到端实验结果。评估了两种数据类型：float32和float16。

学习加速器（VDLA）——它从先前的加速器提案[12, 21, 27]中提炼出特性，融入极简硬件架构——以展示TVM生成针对专用加速器的高效调度能力。图20展示了VDLA架构的高层硬件组织。VDLA被编程为张量处理器，以高效执行计算密集型操作（例如矩阵乘法、高维卷积）。它能执行加载/存储操作，将分块的3维张量从DRAM移入SRAM的连续区域。它还提供专用的片上存储器，用于存储网络参数、层输入（窄数据类型）和层输出（宽数据类型）。最后，VDLA提供对连续加载、计算和存储的显式同步控制，以最大化内存与计算操作之间的重叠。

方法。我们在一个低功耗的PYNQ板上实现了VDLA设计，该板包含一个主频为667MHz的ARM Cortex A9双核CPU和一个基于Artix-7的FPGA架构。在这些有限的FPGA资源上，我们实现了一个主频为200MHz的 16×16 矩阵向量单元，每个周期执行8位值的乘积并将其累加到一个32位寄存器中。该VDLA设计的理论峰值吞吐量约为102.4GOPS/s。我们分配了32kB资源用于激活存储，32kB用于参数存储，32kB用于微码缓冲区，以及128kB用于寄存器文件。这些片上缓冲区远不足以为一层ResNet提供足够的片上存储，因此为有效的内存重用和延迟隐藏提供了一个案例研究。

我们为VDLA构建了一个驱动程序库，其C运行时API能够构建指令并将其推送到目标加速器执行。随后，我们的代码生成算法将加速器程序转换为一连串对运行时API的调用。添加专用加速器后端仅需在Python中编写 $\sim 2k$ 行代码。

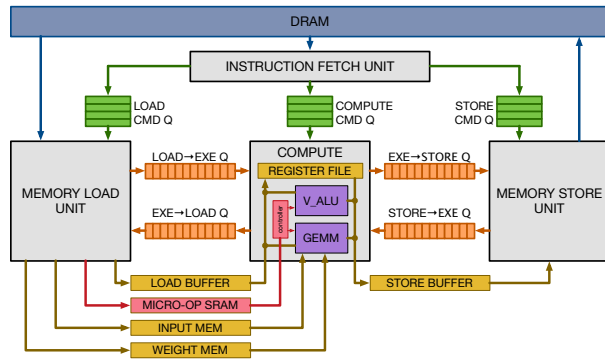


Figure 20: VDLA Hardware design overview.

End-to-End ResNet Evaluation. We used TVM to generate ResNet inference kernels on the PYNQ platform and offloaded as many layers as possible to VDLA. We also used it to generate both schedules for the CPU only and CPU+FPGA implementation. Due to its shallow convolution depth, the first ResNet convolution layer could not be efficiently offloaded on the FPGA and was instead computed on the CPU. All other convolution layers in ResNet, however, were amenable to efficient offloading. Operations like residual layers and activations were also performed on the CPU since VDLA does not support these operations.

Figure 21 breaks down ResNet inference time into CPU-only execution and CPU+FPGA execution. Most computation was spent on the convolution layers that could be offloaded to VDLA. For those convolution layers, the achieved speedup was $40\times$. Unfortunately, due to Amdahl’s law, the overall performance of the FPGA accelerated system was bottlenecked by the sections of the workload that had to be executed on the CPU. We envision that extending the VDLA design to support these other operators will help reduce cost even further. This FPGA-based experiment showcases TVM’s ability to adapt to new architectures and the hardware intrinsics they expose.

7 Related Work

Deep learning frameworks [3, 4, 7, 9] provide convenient interfaces for users to express DL workloads and deploy them easily on different hardware back-ends. While existing frameworks currently depend on vendor-specific tensor operator libraries to execute their workloads, they can leverage TVM’s stack to generate optimized code for a larger number of hardware devices.

High-level computation graph DSLs are a typical way to represent and perform high-level optimizations. Tensorflow’s XLA [3] and the recently introduced DLVM [45] fall into this category. The representations

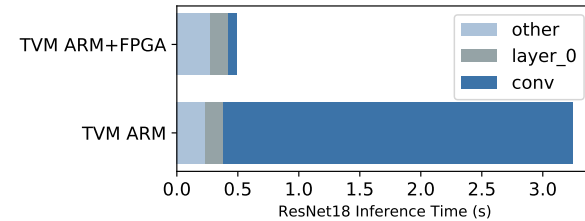


Figure 21: We offloaded convolutions in the ResNet workload to an FPGA-based accelerator. The grayed-out bars correspond to layers that could not be accelerated by the FPGA and therefore had to run on the CPU. The FPGA provided a $40\times$ acceleration on offloaded convolution layers over the Cortex A9.

of computation graphs in these works are similar, and a high-level computation graph DSL is also used in this paper. While graph-level representations are a good fit for high-level optimizations, they are too high level to optimize tensor operators under a diverse set of hardware back-ends. Prior work relies on specific lowering rules to directly generate low-level LLVM or resorts to vendor-crafted libraries. These approaches require significant engineering effort for each hardware back-end and operator-variant combination.

Halide [32] introduced the idea of separating computing and scheduling. We adopt Halide’s insights and reuse its existing useful scheduling primitives in our compiler. Our tensor operator scheduling is also related to other work on DSL for GPUs [18, 24, 36, 37] and polyhedral-based loop transformation [6, 43]. TACO [23] introduces a generic way to generate sparse tensor operators on CPU. Weld [30] is a DSL for data processing tasks. We specifically focus on solving the new scheduling challenges of DL workloads for GPUs and specialized accelerators. Our new primitives can potentially be adopted by the optimization pipelines in these works.

High-performance libraries such as ATLAS [46] and FFTW [15] use auto-tuning to get the best performance. Tensor comprehension [42] applied black-box auto-tuning together with polyhedral optimizations to optimize CUDA kernels. OpenTuner [5] and existing hyper parameter-tuning algorithms [26] apply domain-agnostic search. A predefined cost model is used to automatically schedule image processing pipelines in Halide [29]. TVM’s ML model uses effective domain-aware cost modeling that considers program structure. The based distributed schedule optimizer scales to a larger search space and can find state-of-the-art kernels on a large range of supported back-ends. More importantly, we provide an end-to-end stack that can take descriptions directly from DL frameworks and jointly optimize together with the graph-level stack.

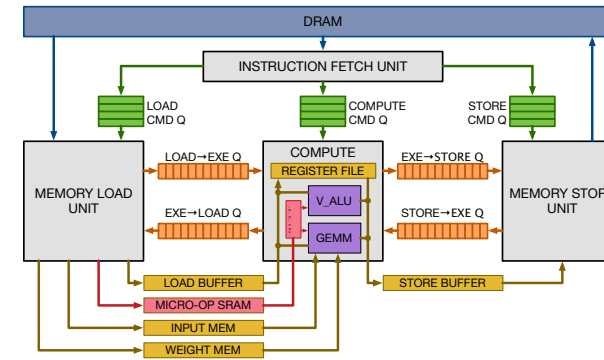


图20: VDLA硬件设计概览。

端到端ResNet评估。我们使用TVM在PYNQ平台上生成ResNet推理内核，并尽可能将各层卸载到VDLA。同时，我们还用它生成了仅CPU和CPU+FPGA两种实现方案的时间表。由于第一个ResNet卷积层的卷积深度较浅，无法在FPGA上高效卸载，因此改在CPU上计算。然而，ResNet中的所有其他卷积层都适合高效卸载。由于VDLA不支持残差层和激活等操作，这些操作也在CPU上执行。

图21将ResNet推理时间分解为仅CPU执行和CPU+FPGA执行。大部分计算时间消耗在可卸载至VDLA的卷积层上。针对这些卷积层，实现的加速比为 $40\times$ 。遗憾的是，受阿姆达尔定律制约，FPGA加速系统的整体性能受限于必须在CPU上执行的工作负载部分。我们设想通过扩展VDLA设计以支持其他算子，将有助于进一步降低成本。这项基于FPGA的实验展示了TVM适应新架构及其所暴露硬件内在函数的能力。

7 相关工作

深度学习框架[3, 4, 7, 9]为用户提供了便捷的接口，使其能够轻松表达深度学习工作负载，并在不同的硬件后端上进行部署。虽然现有框架目前依赖供应商特定的张量算子库来执行其工作负载，但它们可以利用TVM的堆栈为更多硬件设备生成优化代码。

高级计算图DSL是表示和执行高级优化的典型方式。TensorFlow的XLA[3]和最近推出的DLVM[45]都属于这一类别。这些表示

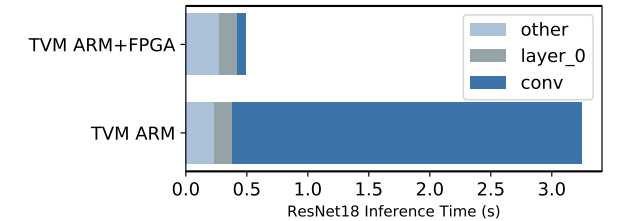


图21: 我们将ResNet工作负载中的卷积运算卸载到基于FPGA的加速器上。灰色柱状图对应无法由FPGA加速、因此必须在CPU上运行的层。对于已卸载的卷积层，FPGA相比Cortex A9实现了40倍的加速效果。

这些工作中的计算图设计相似，本文也采用了一种高级计算图DSL。虽然图级表示非常适合高级优化，但对于多样化硬件后端下的张量算子优化而言，其抽象层级过高。先前研究依赖特定的降级规则直接生成低级LLVM代码，或采用供应商预制的函数库。这些方法需要为每个硬件后端与算子变体的组合投入大量工程开发工作。

Halide [32] 引入了将计算与调度分离的思想。我们采纳了Halide的见解，并在我们的编译器中复用了其现有的有用调度原语。我们的张量算子调度也与其他关于GPU的DSL研究[18, 24, 36, 37]以及基于多面体的循环变换[6, 43]相关。TACO [23] 提出了一种在CPU上生成稀疏张量算子的通用方法。Weld [30] 是一种用于数据处理任务的DSL。我们特别专注于解决针对GPU和专用加速器的深度学习工作负载所带来的新调度挑战。我们的新原语有可能被这些工作中的优化流程所采纳。

诸如ATLAS [46]和FFTW [15]等高性能库采用自动调优技术以获得最佳性能。张量理解[42]将黑盒自动调优与多面体优化相结合，以优化CUDA内核。OpenTuner [5]和现有的超参数调优算法[26]采用领域无关的搜索方法。Halide [29]使用预定义的成本模型自动调度图像处理流水线。TVM的机器学习模型采用有效的领域感知成本建模，该建模考虑了程序结构。基于分布式调度优化器可扩展至更大的搜索空间，并能在广泛支持的后端上找到最先进的内核。更重要的是，我们提供了一个端到端的堆栈，可以直接从深度学习框架获取描述，并与图级堆栈联合优化。

Despite the emerging popularity of accelerators for deep learning [11, 21], it remains unclear how a compilation stack can be built to effectively target these devices. The VDLA design used in our evaluation provides a generic way to summarize the properties of TPU-like accelerators and enables a concrete case study on how to compile code for accelerators. Our approach could potentially benefit existing systems that compile deep learning to FPGA [34, 40], as well. This paper provides a generic solution to effectively target accelerators via tensorization and compiler-driven latency hiding.

8 Conclusion

We proposed an end-to-end compilation stack to solve fundamental optimization challenges for deep learning across a diverse set of hardware back-ends. Our system includes automated end-to-end optimization, which is historically a labor-intensive and highly specialized task. We hope this work will encourage additional studies of end-to-end compilation approaches and open new opportunities for DL system software-hardware co-design techniques.

Acknowledgement

We would like to thank Ras Bodik, James Bornholt, Xi Wang, Tom Anderson and Qiao Zhang for their thorough feedback on earlier versions of this paper. We would also like to thank members of Sampa, SAMPL and Systems groups at the Allen School for their feedback on the work and manuscript. We would like to thank the anonymous OSDI reviewers, and our shepherd, Ranjita Bhagwan, for helpful feedbacks. This work was supported in part by a Google PhD Fellowship for Tianqi Chen, ONR award #N00014-16-1-2795, NSF under grants CCF-1518703, CNS-1614717, and CCF-1723352, and gifts from Intel (under the CAPA program), Oracle, Huawei and anonymous sources.

References

[1] NVIDIA Tesla V100 GPU Architecture: The World’s Most Advanced Data Center GPU, 2017.

[2] ABADI, M., AGARWAL, A., BARHAM, P., BREVDO, E., CHEN, Z., CITRO, C., CORRADO, G. S., DAVIS, A., DEAN, J., DEVIN, M., GHEMAWAT, S., GOODFELLOW, I., HARP, A., IRVING, G., ISARD, M., JIA, Y., JOZEFOWICZ, R., KAISER, L., KUDLUR, M., LEVENBERG, J., MANÉ, D., MONGA, R., MOORE, S., MURRAY, D., OLAH, C., SCHUSTER, M., SHLENS, J., STEINER, B., SUTSKEVER, I., TALWAR, K., TUCKER, P., VANHOUCKE, V., VASUDEVAN, V., VIÉGAS, F., VINYALS, O., WARDEN, P., WATTENBERG, M., WICKE, M., YU, Y., AND ZHENG, X. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.

[3] ABADI, M., BARHAM, P., CHEN, J., CHEN, Z., DAVIS, A., DEAN, J., DEVIN, M., GHEMAWAT, S., IRVING, G., ISARD, M., KUDLUR, M., LEVENBERG, J., MONGA, R., MOORE, S., MURRAY, D. G., STEINER, B., TUCKER, P., VASUDEVAN, V., WARDEN, P., WICKE, M., YU, Y., AND ZHENG, X. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (2016), pp. 265–283.

[4] AGARWAL, A., AKCHURIN, E., BASOGLU, C., CHEN, G., CYPHERS, S., DROPPA, J., EVERSOLE, A., GUENTER, B., HILLEBRAND, M., HOENS, R., HUANG, X., HUANG, Z., IVANOV, V., KAMENEV, A., KRANEN, P., KUCHAIEV, O., MANOUSEK, W., MAY, A., MITRA, B., NANO, O., NAVARRO, G., ORLOV, A., PADMILAC, M., PARTHASARATHI, H., PENG, B., REZNICHENKO, A., SEIDE, F., SELTZER, M. L., SLANEY, M., STOLCKE, A., WANG, Y., WANG, H., YAO, K., YU, D., ZHANG, Y., AND ZWEIG, G. An introduction to computational networks and the computational network toolkit. Tech. Rep. MSR-TR-2014-112, August 2014.

[5] ANSEL, J., KAMIL, S., VEERAMACHANENI, K., RAGAN-KELLEY, J., BOSBOOM, J., O’REILLY, U.-M., AND AMARASINGHE, S. Opentuner: An extensible framework for program autotuning. In *International Conference on Parallel Architectures and Compilation Techniques* (Edmonton, Canada, August 2014).

[6] BAGHDADI, R., BEAUGNON, U., COHEN, A., GROSSER, T., KRUSE, M., REDDY, C., VERDOOLAEGE, S., BETTS, A., DONALDSON, A. F., KETEMA, J., ABSAR, J., HAASTREGT, S. V., KRAVETS, A., LOKHMOTOV, A., DAVID, R., AND HAJIYEV, E. Pencil: A platform-neutral compute intermediate language for accelerator programming. In *Proceedings of the 2015 International Conference on Parallel Architecture and Compilation (PACT)* (Washington, DC, USA, 2015), PACT ’15, IEEE Computer Society, pp. 138–149.

[7] BASTIEN, F., LAMBLIN, P., PASCANU, R., BERGSTRA, J., GOODFELLOW, I. J., BERGERON, A., BOUCHARD, N., AND BENGIO, Y. Theano: new features and speed improvements. Deep Learning and Unsupervised Feature Learning NIPS 2012 Workshop, 2012.

[8] CHEN, T., AND GUESTSTRIN, C. Xgboost: A scalable tree boosting system. In *Proceedings of the 22Nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (New York, NY, USA, 2016), KDD ’16, ACM, pp. 785–794.

[9] CHEN, T., LI, M., LI, Y., LIN, M., WANG, N., WANG, M., XIAO, T., XU, B., ZHANG, C., , AND ZHANG, Z. MXNet: A flexible and efficient machine learning library for heterogeneous distributed systems. In *Neural Information Processing Systems, Workshop on Machine Learning Systems (LearningSys’15)* (2015).

[10] CHEN, T.-F., AND BAER, J.-L. Effective hardware-based data prefetching for high-performance processors. *IEEE Transactions on Computers* 44, 5 (May 1995), 609–623.

[11] CHEN, Y., LUO, T., LIU, S., ZHANG, S., HE, L., WANG, J., LI, L., CHEN, T., XU, Z., SUN, N., AND TEMAM, O. Dadiannao: A machine-learning supercomputer. In *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture* (Washington, DC, USA, 2014), MICRO-47, IEEE Computer Society, pp. 609–622.

[12] CHEN, Y.-H., EMER, J., AND SZE, V. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. In *Proceedings of the 43rd International Symposium on Computer Architecture* (Piscataway, NJ, USA, 2016), ISCA ’16, IEEE Press, pp. 367–379.

[13] COURBARIAUX, M., BENGIO, Y., AND DAVID, J. Binaryconnect: Training deep neural networks with binary weights during propagations. *CoRR abs/1511.00363* (2015).

尽管深度学习加速器日益流行[11, 21], 但如何构建编译栈以有效适配这些设备仍不明确。我们评估中使用的VDLA设计提供了一种通用方法来概括类TPU加速器的特性, 并为如何为加速器编译代码提供了具体案例研究。我们的方法也可能有益于现有将深度学习编译到FPGA的系统[34,40]。本文通过张量化和编译器驱动的延迟隐藏, 为有效适配加速器提供了通用解决方案。

8 结论

我们提出了一个端到端的编译栈, 以解决跨多种硬件后端进行深度学习时的基本优化挑战。我们的系统包含自动化的端到端优化, 这在历史上是一项劳动密集且高度专业化的任务。我们希望这项工作能够鼓励更多关于端到端编译方法的研究, 并为深度学习系统软件与硬件的协同设计技术开辟新的机遇。

致谢

我们要感谢Ras Bodik、James Bornholt、Xi Wang、Tom Anderson和Qiao Zhang对本文早期版本提出的详尽反馈。同时, 也要感谢艾伦学院Sampa、SAMPL和Systems小组的成员们对此项研究及手稿的宝贵意见。我们还要感谢匿名的OSDI审稿人, 以及我们的指导人Ranjita Bhagwan, 他们提供了有益的反馈。此项工作部分得到了Tianqi Chen的谷歌博士奖学金、ONR奖项#N00014-16-1-2795、NSF资助CCF-1518703、CNS-1614717和CCF-1723352, 以及英特尔(通过CAPA项目)、Oracle、华为和匿名来源的捐赠支持。

参考文献

[1] NVIDIA Tesla V100 GPU架构: 全球最先进的数据中心GPU, 2017年。

[2] ABADI, M., AGARWAL, A., BARHAM, P., BREVDO, E., CHEN, Z., CITRO, C., CORRADO, G. S., DAVIS, A., DEAN, J., DEVIN, M., GHEMAWAT, S., GOODFELLOW, I., HARP, A., IRVING, G., ISARD, M., JIA, Y., JOZEFOWICZ, R., KAISER, L., KUDLUR, M., LEVENBERG, J., MANÉ, D., MONGA, R., MOORE, S., MURRAY, D., OLAH, C., SCHUSTER, M., SHLENS, J., STEINER, B., SUTSKEVER, I., TALWAR, K., TUCKER, P., VANHOUCKE, V., VASUDEVAN, V., VIÉGAS, F., VINYALS, O., WARDEN, P., WATTENBERG, M., WICKE, M., YU, Y., AND ZHENG, X. TensorFlow: 异构系统上的大规模机器学习, 2015。软件可从 tensorflow.org 获取。

[3] ABADI, M., BARHAM, P., CHEN, J., CHEN, Z., DAVIS, A., DEAN, J., DEVIN, M., GHEMAWAT, S., IRVING, G., ISARD, M., KUDLUR, M., LEVENBERG, J., MONGA, R., MOORE, S., MURRAY, D. G., STEINER, B., TUCKER, P., VASUDEVAN, V., WARDEN, P., WICKE, M., YU, Y., 与 ZHENG, X. TensorFlow: 一个用于大规模机器学习的系统。于 *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (2016), 第265–283页。

[4] AGARWAL, A., AKCHURIN, E., BASOGLU, C., CHEN, G., CYPHERS, S., DROPPA, J., EVERSOLE, A., GUENTER, B., HILLEBRAND, M., HOENS, R., HUANG, X., HUANG, Z., IVANOV, V., KAMENEV, A., KRANEN, P., KUCHAIEV, O., MANOUSEK, W., MAY, A., MITRA, B., NANO, O., NAVARRO, G., ORLOV, A., PADMILAC, M., PARTHASARATHI, H., PENG, B., REZNICHENKO, A., SEIDE, F., SELTZER, M. L., SLANEY, M., STOLCKE, A., WANG, Y., WANG, H., YAO, K., YU, D., ZHANG, Y., AND ZWEIG, G. 计算网络与计算网络工具包简介。技术报告 MSR-TR-2014-112, 2014年8月。

[5] ANSEL, J., KAMIL, S., VEERAMACHANENI, K., RAGAN-KELLEY, J., BOSBOOM, J., O’REILLY, U.-M., AND AMARASINGHE, S. Opentuner: 一个可扩展的程序自动调优框架。于 *International Conference on Parallel Architectures and Compilation Techniques* (加拿大埃德蒙顿, 2014年8月)。

[6] BAGHDADI, R., BEAUGNON, U., COHEN, A., GROSSER, T., KRUSE, M., REDDY, C., VERDOOLAEGE, S., BETTS, A., DONALDSON, A. F., KETEMA, J., ABSAR, J., HAASTREGT, S. V., KRAVETS, A., LOKHMOTOV, A., DAVID, R., AND HAJIYEV, E. Pencil: 一种用于加速器编程的平台中立计算中间语言。发表于 *Proceedings of the 2015 International Conference on Parallel Architecture and Compilation (PACT)* (华盛顿特区, 美国, 2015), PACT ’15, IEEE计算机协会, 第138–149页。

[7] BASTIEN, F., LAMBLIN, P., PASCANU, R., BERGSTRA, J., GOODFELLOW, I. J., BERGERON, A., BOUCHARD, N., 与 BENGIO, Y. Theano: 新特性与性能提升。深度学习与无监督特征学习 NIPS 2012 研讨会, 2012。

[8] CHEN, T., 与 GUESTSTRIN, C. Xgboost: 一种可扩展的树提升系统。收录于 *Proceedings of the 22Nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (美国纽约州纽约市, 2016), KDD ’16, ACM, 第785–794页。

[9] CHEN, T., LI, M., LI, Y., LIN, M., WANG, N., WANG, M., XIAO, T., XU, B., ZHANG, C., AND ZHANG, Z. MXNet: 面向异构分布式系统的灵活高效机器学习库。于 *Neural Information Processing Systems, Workshop on Machine Learning Systems (LearningSys’15)* (2015)。

[10] CHEN, T.-F. 与 BAER, J.-L. 面向高性能处理器的有效硬件数据预取技术。 *IEEE Transactions on Computers* 44, 5 (1995年5月), 609–623。

[11] 陈云霄、罗韬、刘世杰、张帅、何磊、王健、李磊、陈天石、徐志伟、孙凝晖、TEMAM, O.。Dadiannao: 一种机器学习超级计算机。发表于*Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture* (华盛顿特区, 美国, 2014), MICRO-47, IEEE计算机学会, 第609–622页。

[12] CHEN, Y.-H., EMER, J., AND SZE, V. Eyeriss: 一种用于卷积神经网络能效数据流的空间架构。于 *Proceedings of the 43rd International Symposium on Computer Architecture* (新泽西州皮斯卡塔维, 美国, 2016), ISCA ’16, IEEE出版社, 第367–379页。[13] COURBARIAUX, M., BENGIO, Y., AND DAVID, J. BinaryConnect: 在传播过程中使用二元权重训练深度神经网络。 *CoRR abs/1511.00363* (2015)。

- [14] EGGERS, S. J., EMER, J. S., LEVY, H. M., LO, J. L., STAMM, R. L., AND TULLSEN, D. M. Simultaneous multithreading: a platform for next-generation processors. *IEEE Micro* 17, 5 (Sept 1997), 12–19.
- [15] FRIGO, M., AND JOHNSON, S. G. Fftw: an adaptive software architecture for the fft. In *Acoustics, Speech and Signal Processing, 1998. Proceedings of the 1998 IEEE International Conference on* (May 1998), vol. 3, pp. 1381–1384 vol.3.
- [16] HE, K., ZHANG, X., REN, S., AND SUN, J. Identity mappings in deep residual networks. *arXiv preprint arXiv:1603.05027* (2016).
- [17] HEGARTY, J., BRUNHAVER, J., DeVITO, Z., RAGAN-KELLEY, J., COHEN, N., BELL, S., VASILYEV, A., HOROWITZ, M., AND HANRAHAN, P. Darkroom: Compiling high-level image processing code into hardware pipelines. *ACM Trans. Graph.* 33, 4 (July 2014), 144:1–144:11.
- [18] HENRIKSEN, T., SERUP, N. G. W., ELSMAN, M., HENGLEIN, F., AND OANCEA, C. E. Futhark: Purely functional gpu-programming with nested parallelism and in-place array updates. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (New York, NY, USA, 2017), PLDI 2017, ACM, pp. 556–571.
- [19] HOWARD, A. G., ZHU, M., CHEN, B., KALENICHENKO, D., WANG, W., WEYAND, T., ANDREETTO, M., AND ADAM, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *CoRR abs/1704.04861* (2017).
- [20] JOUPPI, N. P. Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers. In *[1990] Proceedings. The 17th Annual International Symposium on Computer Architecture* (May 1990), pp. 364–373.
- [21] JOUPPI, N. P., YOUNG, C., PATIL, N., PATTERSON, D., AGRAWAL, G., BAJWA, R., BATES, S., BHATIA, S., BODEN, N., BORCHERS, A., BOYLE, R., CANTIN, P.-L., CHAO, C., CLARK, C., CORIELL, J., DALEY, M., DAU, M., DEAN, J., GELB, B., GHAEMMAGHAMI, T. V., GOTTIPATI, R., GULLAND, W., HAGMANN, R., HO, C. R., HOGBERG, D., HU, J., HUNDT, R., HURT, D., IBARZ, J., JAFFEY, A., JAWORSKI, A., KAPLAN, A., KHAITAN, H., KILLEBREW, D., KOCH, A., KUMAR, N., LACY, S., LAUDON, J., LAW, J., LE, D., LEARY, C., LIU, Z., LUCKE, K., LUNDIN, A., MACKEAN, G., MAGGIORE, A., MAHONY, M., MILLER, K., NAGARAJAN, R., NARAYANASWAMI, R., NI, R., NIX, K., NORRIE, T., OMERNICK, M., PENUKONDA, N., PHELPS, A., ROSS, J., ROSS, M., SALEK, A., SAMADIANI, E., SEVERN, C., SIZIKOV, G., SNELHAM, M., SOUTER, J., STEINBERG, D., SWING, A., TAN, M., THORSON, G., TIAN, B., TOMA, H., TUTTLE, E., VASUDEVAN, V., WALTER, R., WANG, W., WILCOX, E., AND YOON, D. H. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture* (New York, NY, USA, 2017), ISCA ’17, ACM, pp. 1–12.
- [22] KIRKPATRICK, S., GELATT, C. D., AND VECCHI, M. P. Optimization by simulated annealing. *Science* 220, 4598 (1983), 671–680.
- [23] KJOLSTAD, F., KAMIL, S., CHOU, S., LUGATO, D., AND AMARASINGHE, S. The tensor algebra compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (Oct. 2017), 77:1–77:29.
- [24] KLÖCKNER, A. Loo.py: transformation-based code generation for GPUs and CPUs. In *Proceedings of ARRAY ’14: ACM SIGPLAN Workshop on Libraries, Languages, and Compilers for Array Programming* (Edinburgh, Scotland., 2014), Association for Computing Machinery.
- [25] LAVIN, A., AND GRAY, S. Fast algorithms for convolutional neural networks. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016* (2016), pp. 4013–4021.
- [26] LI, L., JAMIESON, K. G., DeSALVO, G., ROSTAMIZADEH, A., AND TALWALKAR, A. Efficient hyperparameter optimization and infinitely many armed bandits. *CoRR abs/1603.06560* (2016).
- [27] LIU, D., CHEN, T., LIU, S., ZHOU, J., ZHOU, S., TEMAN, O., FENG, X., ZHOU, X., AND CHEN, Y. Pudiannao: A polyvalent machine learning accelerator. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems* (New York, NY, USA, 2015), ASPLOS ’15, ACM, pp. 369–381.
- [28] MNIH, V., KAVUKCUOGLU, K., SILVER, D., RUSU, A. A., VENESS, J., BELLEMARE, M. G., GRAVES, A., RIEDMILLER, M., FIDJELAND, A. K., OSTROVSKI, G., ET AL. Human-level control through deep reinforcement learning. *Nature* 518, 7540 (2015), 529.
- [29] MULLAPUDI, R. T., ADAMS, A., SHARLET, D., RAGAN-KELLEY, J., AND FATAHALIAN, K. Automatically scheduling halide image processing pipelines. *ACM Trans. Graph.* 35, 4 (July 2016), 83:1–83:11.
- [30] PALKAR, S., THOMAS, J. J., NARAYANAN, D., SHANBHAG, A., PALAMUTTAM, R., PIRK, H., SCHWARZKOPF, M., AMARASINGHE, S. P., MADDEN, S., AND ZAHARIA, M. Weld: Re-thinking the interface between data-intensive applications. *CoRR abs/1709.06416* (2017).
- [31] RADFORD, A., METZ, L., AND CHINTALA, S. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434* (2015).
- [32] RAGAN-KELLEY, J., BARNES, C., ADAMS, A., PARIS, S., DURAND, F., AND AMARASINGHE, S. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (New York, NY, USA, 2013), PLDI ’13, ACM, pp. 519–530.
- [33] RASTEGARI, M., ORDONEZ, V., REDMON, J., AND FARHADI, A. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European Conference on Computer Vision* (2016), Springer, pp. 525–542.
- [34] SHARMA, H., PARK, J., MAHAJAN, D., AMARO, E., KIM, J. K., SHAO, C., MISHRA, A., AND ESMAELIZADEH, H. From high-level deep neural models to fpgas. In *Microarchitecture (MICRO), 2016 49th Annual IEEE/ACM International Symposium on* (2016), IEEE, pp. 1–12.
- [35] SMITH, J. E. Decoupled access/execute computer architectures. In *Proceedings of the 9th Annual Symposium on Computer Architecture* (Los Alamitos, CA, USA, 1982), ISCA ’82, IEEE Computer Society Press, pp. 112–119.
- [36] STEUWER, M., REMMELG, T., AND DUBACH, C. Lift: A functional data-parallel ir for high-performance gpu code generation. In *Proceedings of the 2017 International Symposium on Code Generation and Optimization* (Piscataway, NJ, USA, 2017), CGO ’17, IEEE Press, pp. 74–85.
- [37] SUJEETH, A. K., LEE, H., BROWN, K. J., CHAFI, H., WU, M., ATREYA, A. R., OLUKOTUN, K., ROMPF, T., AND ODEFSKY, M. Optiml: An implicitly parallel domain-specific language for machine learning. In *Proceedings of the 28th International Conference on International Conference on Machine Learning* (USA, 2011), ICML ’11, pp. 609–616.
- [38] TAI, K. S., SOCHER, R., AND MANNING, C. D. Improved semantic representations from tree-structured long short-term memory networks. *arXiv preprint arXiv:1503.00075* (2015).

[14] EGGERS, S. J., EMER, J. S., LEVY, H. M., LO, J. L., STAMM, R. L., AND TULLSEN, D. M. 同步多线程：下一代处理器的平台。 *IEEE Micro* 17, 5 (1997年9月), 12–19. [15] FRIGO, M., AND JOHNSON, S. G. FFTW: 一种用于FFT的自适应软件架构。载于 *Acoustics, Speech and Signal Processing, 1998. Proceedings of the 1998 IEEE International Conference on* (1998年5月), 第3卷, 第1381–1384页, 第3卷. [16] HE, K., ZHANG, X., REN, S., AND SUN, J. 深度残差网络中的恒等映射。 *arXiv preprint arXiv:1603.05027* (2016). [17] HEGARTY, J., BRUNHAVER, J., DEVITO, Z., RAGAN-KELLEY, J., COHEN, N., BELL, S., VASILYEV, A., HOROWITZ, M., AND HANRAHAN, P. Darkroom: 将高级图像处理代码编译为硬件流水线。 *ACM Trans. Graph.* 33, 4 (2014年7月), 144:1–144:11. [18] HENRIKSEN, T., SERUP, N. G. W., ELSMAN, M., HENGLEIN, F., AND OANCEA, C. E. Futhark: 具有嵌套并行性和就地数组更新的纯函数式GPU编程。载于 *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (美国纽约州纽约市, 2017), PLDI 2017, ACM, 第556–571页. [19] HOWARD, A. G., ZHU, M., CHEN, B., KALENICHENKO, D., WANG, W., WEYAND, T., ANDREETTO, M., AND ADAM, H. MobileNets: 用于移动视觉应用的高效卷积神经网络。 *CoRR abs/1704.04861* (2017). [20] JOUPPI, N. P. 通过添加小型全相联缓存和预取缓冲区改进直接映射缓存性能。载于 *[1990] Proceedings. The 17th Annual International Symposium on Computer Architecture* (1990年5月), 第364–373页. [21] JOUPPI, N. P., YOUNG, C., PATIL, N., PATTERSON, D., AGRAWAL, G., BAJWA, R., BATES, S., BHATIA, S., BODEN, N., BORCHERS, A., BOYLE, R., CANTIN, P.-L., CHAO, C., CLARK, C., CORIELL, J., DALEY, M., DAU, M., DEAN, J., GELB, B., GHAEMMAGHAMI, T. V., GOTTIPATI, R., GULLAND, W., HAGMANN, R., HO, C. R., HOGBERG, D., HU, J., HUNDT, R., HURT, D., IBARZ, J., JAFFEY, A., JAWORSKI, A., KAPLAN, A., KHAITAN, H., KILLEBREW, D., KOCH, A., KUMAR, N., LACY, S., LAUDON, J., LAW, J., LE, D., LEARY, C., LIU, Z., LUCKE, K., LUNDIN, A., KAPLAN, A., KHAITAN, H., KILLEBREW, D., KOCH, A., KUMAR, N., LACY, S., LAUDON, J., LAW, J., LE, D., LEARY, C., LIU, Z., LUCKE, K., LUNDIN, A., MACKEAN, G., MAGGIORE, A., MAHONY, M., MILLER, K., NAGARAJAN, R., NARAYANASWAMI, R., NI, R., NIX, K., NORRIE, T., OMERNICK, M., PENUKONDA, N., PHELPS, A., ROSS, J., ROSS, M., SALEK, A., SAMADIANI, E., SEVERN, C., SIZIKOV, G., SNELHAM, M., SOUTER, J., STEINBERG, D., SWING, A., TAN, M., THORSON, G., TIAN, B., TOMA, H., TUTTLE, E., VASUDEVAN, V., WALTER, R., WANG, W., WILCOX, E., AND YOON, D. H. 张量处理单元在数据中心内的性能分析。载于 *Proceedings of the 44th Annual International Symposium on Computer Architecture* (美国纽约州纽约市, 2017), ISCA ’17, ACM, 第1–12页. [22] KIRKPATRICK, S., GELATT, C. D., AND VECCHI, M. P. 模拟退火优化。 *Science* 220, 4598 (1983), 671–680. [23] KJOLSTAD, F., KAMIL, S., CHOU, S., LUGATO, D., AND AMARASINGHE, S. 张量代数编译器。 *Proc. ACM Program. Lang.* 1, OOPSLA (2017年10月), 77:1–77:29. [24] KLÖCKNER, A. Loo.py: 基于转换的GPU和CPU代码生成。载于 *Proceedings of ARRAY ’14: ACM SIGPLAN Workshop on Libraries, Languages, and Compilers for Array Programming* (苏格兰爱丁堡, 2014), 计算机协会. [25] LAVIN, A., AND GRAY, S. 卷积神经网络的快速算法。载于 *2016 IEEE Conference on Computer Vision*

and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016 (2016年, 第4013–4021页。

[26] LI, L., JAMIESON, K. G., DeSALVO, G., ROSTAMIZADEH, A., AND TALWALKAR, A. 高效超参数优化与无限臂老虎机。 *CoRR abs/1603.06560* (2016). [27] LIU, D., CHEN, T., LIU, S., ZHOU, J., ZHOU, S., TEMAN, O., FENG, X., ZHOU, X., AND CHEN, Y. 普适神经网络：一种多用途机器学习加速器。载于 *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems* (美国纽约州纽约市, 2015), ASPLOS ’15, ACM, 第369–381页. [28] MNIH, V., KAVUKCUOGLU, K., SILVER, D., RUSU, A. A., VENESS, J., BELLEMARE, M. G., GRAVES, A., RIEDMILLER, M., FIDJELAND, A. K., OSTROVSKI, G., 等。通过深度强化学习实现人类水平控制。 *Nature* 518, 7540 (2015), 第529页. [29] MULLAPUDI, R. T., ADAMS, A., SHARLET, D., RAGAN-KELLEY, J., AND FATAHALIAN, K. 自动调度Halide图像处理流水线。 *ACM Trans. Graph.* 35, 4 (2016年7月), 第83:1–83:11页. [30] PALKAR, S., THOMAS, J. J., NARAYANAN, D., SHANBHAG, A., PALAMUTTAM, R., PIRK, H., SCHWARZKOPF, M., AMARASINGHE, S. P., MADDEN, S., AND ZAHARIA, M. Weld: 重新思考数据密集型应用间的接口。 *CoRR abs/1709.06416* (2017). [31] RADFORD, A., METZ, L., AND CHINTALA, S. 使用深度卷积生成对抗网络进行无监督表示学习。 *arXiv preprint arXiv:1511.06434* (2015). [32] RAGAN-KELLEY, Y. J., BARNES, C., ADAMS, A., PARIS, S., DURAND, F., AND AMARASINGHE, S. Halide: 一种用于优化图像处理流水线中并行性、局部性和重计算的语言与编译器。载于 *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (美国纽约州纽约市, 2013), PLDI ’13, ACM, 第519–530页. [33] RASTEGARI, M., ORDONEZ, V., REDMON, J., AND FARHADI, A. XNor-Net: 使用二值卷积神经网络进行ImageNet分类。载于 *European Conference on Computer Vision* (2016), Springer, 第525–542页. [34] SHARMA, H., PARK, J., MAHAJAN, D., AMARO, E., KIM, J. K., SHAO, C., MISHRA, A., AND ESMAELIZADEH, H. 从高层次深度神经网络模型到FPGA。载于 *Microarchitecture (MICRO), 2016 49th Annual IEEE/ACM International Symposium on* (2016), IEEE, 第1–12页. [35] SMITH, J. E. 解耦访问/执行计算机体系结构。载于 *Proceedings of the 9th Annual Symposium on Computer Architecture* (美国加利福尼亚州洛杉矶阿拉米托斯, 1982), ISCA ’82, IEEE计算机学会出版社, 第112–119页. [36] STEUWER, M., REMMELG, T., AND DUBACH, C. Lift: 一种用于高性能GPU代码生成的功能性数据并行中间表示。载于 *Proceedings of the 2017 International Symposium on Code Generation and Optimization* (美国新泽西州皮斯卡塔维, 2017), CGO ’17, IEEE出版社, 第74–85页. [37] SUJEETH, A. K., LEE, H., BROWN, K. J., CHAFI, H., WU, M., ATREYA, A. R., OLUKOTUN, K., ROMPF, T., AND ODEFSKY, M. OptiML: 一种用于机器学习的隐式并行领域特定语言。载于 *Proceedings of the 28th International Conference on International Conference on Machine Learning* (美国, 2011), ICML ’11, 第609–616页. [38] TAI, K. S., SOCHER, R., AND MANNING, C. D. 基于树状结构长短期记忆网络的改进语义表示。 *arXiv preprint arXiv:1503.00075* (2015)。

[39] TULLOCH, A., AND JIA, Y. High performance ultra-low-precision convolutions on mobile devices. *arXiv preprint arXiv:1712.02427* (2017).

[40] UMUROGLU, Y., FRASER, N. J., GAMBARDILLA, G., BLOTT, M., LEONG, P. H. W., JAHRE, M., AND VISSERS, K. A. FINN: A framework for fast, scalable binarized neural network inference. *CoRR abs/1612.07119* (2016).

[41] VASILACHE, N. personal communication.

[42] VASILACHE, N., ZINENKO, O., THEODORIDIS, T., GOYAL, P., DEVITO, Z., MOSES, W. S., VERDOOLAEGE, S., ADAMS, A., AND COHEN, A. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *CoRR abs/1802.04730* (2018).

[43] VERDOOLAEGE, S., CARLOS JUEGA, J., COHEN, A., IGNACIO GÓMEZ, J., TENLLADO, C., AND CATTHOOR, F. Polyhedral parallel code generation for cuda. *ACM Trans. Archit. Code Optim.* 9, 4 (Jan. 2013), 54:1–54:23.

[44] VOLKOV, V. *Understanding Latency Hiding on GPUs*. PhD thesis, University of California at Berkeley, 2016.

[45] WEI, R., ADVE, V., AND SCHWARTZ, L. DlvM: A modern compiler infrastructure for deep learning systems. *CoRR abs/1711.03016* (2017).

[46] WHALEY, R. C., AND DONGARRA, J. J. Automatically tuned linear algebra software. In *Proceedings of the 1998 ACM/IEEE Conference on Supercomputing* (Washington, DC, USA, 1998), SC '98, IEEE Computer Society, pp. 1–27.

[47] WILLIAMS, S., WATERMAN, A., AND PATTERSON, D. Roofline: An insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (Apr. 2009), 65–76.

[48] ZAREMBA, W., SUTSKEVER, I., AND VINYALS, O. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329* (2014).

[39] TULLOCH, A. 与 JIA, Y. 在移动设备上实现高性能超低精度卷积。 *arXiv preprint arXiv:1712.02427* (2017)。

[40] UMUROGLU, Y., FRASER, N. J., GAMBARDILLA, G., BLOTT, M., LEONG, P. H. W., JAHRE, M. 与 VISSERS, K. A. FINN: 一种用于快速、可扩展二值化神经网络推理的框架。 *CoRR abs/1612.07119* (2016)。

[41] VASILACHE, N. 个人通信。

[42] VASILACHE, N., ZINENKO, O., THEODORIDIS, T., GOYAL, P., DEVITO, Z., MOSES, W. S., VERDOOLAEGE, S., ADAMS, A. 与 COHEN, A. 张量理解: 框架无关的高性能机器学习抽象。 *CoRR abs/1802.04730* (2018)。

[43] VERDOOLAEGE, S., CARLOS JUEGA, J., COHEN, A., IGNACIO GÓMEZ, J., TENLLADO, C. 与 CATTHOOR, F. 面向CUDA的多面体并行代码生成。 *ACM Trans. Archit. Code Optim.* 9, 4 (2013年1月), 54:1–54:23。

[44] VOLKOV, V. *Understanding Latency Hiding on GPUs*。 博士论文, 加州大学伯克利分校, 2016年。

[45] WEI, R., ADVE, V. 与 SCHWARTZ, L. D LVM: 面向深度学习系统的现代编译器基础设施。 *CoRR abs/1711.03016* (2017)。

[46] WHALEY, R. C. 与 DONGARRA, J. J . 自动调优的线性代数软件。 收录于 *Proceedings of the 1998 ACM/IEEE Conference on Supercomputing* (华盛顿特区, 美国, 1998), SC '98, IEEE计算机学会, 第1–27页。

[47] WILLIAMS, S., WATERMAN, A. 与 PATTERSON, D. Roofline: 一种用于多核架构的直观可视化性能模型。 *Commun. ACM* 52, 4 (2009年4月), 65–76。

[48] ZAREMBA, W., SUTSKEVER, I. 与 VINYALS, O. 循环神经网络正则化。 *arXiv preprint arXiv:1409.2329* (2014)。